

Hermite Curves as Trajectory Priors for Vision-Language-Action Models

Qi Lv, Jianming Xing, Zhao Yang, Mingyuan Yao, Yinan Shi, Yawei Jueluo
Mike Zheng Shou, Xiang Deng

Abstract—Despite recent progress in Vision-Language-Action (VLA) models for robotic manipulation, the action chunk remains a weakly structured interface. Existing work typically flatten each chunk into per-timestep controls, relying on implicit data learning that manifests as jagged motion and boundary discontinuities during physical execution. To address these limitations, we introduce Hermite trajectory priors, parameterizing the chunk trajectory as a piecewise cubic Hermite curve defined by endpoint positions and velocities to explicitly enforce smoothness and continuity. We instantiate this fixed operator across discrete autoregressive and continuous generative paradigms via three variants: (1) Hermite Tokens, which predict quantized boundary variables autoregressively; (2) Hermite Scaffold, which decomposes clean actions into a base scaffold and residuals; and (3) Hermite Regularization, which applies the prior strictly as an auxiliary training objective. Across simulation benchmarks and real-robot platforms, Hermite Regularization achieves superior performance among these three variants, improving $\pi_{0.5}$ baseline success rates from 95.9% to 98.7% on LIBERO, 85.7% to 90.9% on LIBERO-plus, and 63.4% to 90.0% across four real-robot tasks without additional inference overhead. Trajectory analyses reveal that explicitly structuring trajectory priors serves most effectively as a learning inductive bias rather than a runtime constraint. Project page is available at <https://aopolin-lv.github.io/Hermite/>.

Index Terms—vision-language-action models, robot learning, action chunking

1 INTRODUCTION

Vision-Language-Action (VLA) models [1], [2], [3], [4] have achieved remarkable success in robotic manipulation by coupling large pre-trained vision-language backbones with action decoders. Among various design choices [5], [6], action chunking, which predicts a short horizon of future actions instead of a single next-step action, has become the prevailing paradigm due to its superior temporal consistency. Despite its widespread adoption, existing VLAs typically model each action chunk as a flat sequence of per-timestep controls [1], [3], [7], [8]. While simple and effective, this representation leaves the action chunk as a weakly structured interface between high-level reasoning and low-level robot control. Thus, how to endow the action chunk with the structure of physical motion remains a challenging and insufficiently explored issue.

A key limitation is that a flat per-timestep representation models the chunk as a sequence of discrete control samples, with no notion of the underlying continuous trajectory that connects them. This single choice has two consequences. (1) Intra-chunk incoherence: since each timestep within a chunk is parameterized independently, optimization naturally relies on pointwise reconstruction losses rather than trajectory-level constraints [9]. Consequently, predictions with similar per-timestep errors receive identical penalties

regardless of their smoothness, leaving jagged intra-chunk motions insufficiently penalized during training. (2) Inter-chunk discontinuity: Across adjacent chunks, closed-loop replanning concatenates independently decoded action sequences without boundary constraints on position or velocity [10], [11]. Thus, transitions between consecutive replanning steps receive no continuity enforcement, leaving abrupt inter-chunk jumps unmitigated during execution. Both failures share the common origin: the flat representation lacks both the temporal smoothness and boundary continuity that physical trajectories inherently require. We thus view this not as a capability ceiling since large backbones can recover smooth motion given enough data, but as a missing inductive bias whose absence is most damaging under distribution shift and physical execution. Figure 1 contrasts existing chunk representations with the proposed prior.

To address these issues, we propose *Hermite trajectory priors* for VLA action modeling. The central idea is to expose the continuous trajectory underlying an action chunk, rather than treating the chunk only as a set of independent per-timestep samples. We represent this trajectory with a piecewise cubic Hermite curve, each segment of which is specified by its boundary conditions, namely the position and velocity at each endpoint. The corresponding discrete chunk is then obtained by applying a fixed linear basis matrix to these boundary parameters. This formulation directly targets the two limitations identified above: (1) It explicitly enforces boundary continuity for inter-chunk replanning. Since the Hermite parameters directly encode endpoint positions and velocities, the boundary conditions governing continuity become an explicit part of the action representation. Assigning matched states to adjacent segment knots biases the decoder toward seamless transitions,

- Qi Lv, Jianming Xing, and Xiang Deng are with School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), China, 518055 (email: lvqi@stu.hit.edu.cn, dengxiang@hit.edu.cn).
- Qi Lv, Zhao Yang, Mingyuan Yao, Yinan Shi, and Yawei Jueluo are with Jiangsu Cytoderm Intelligent Technology Co., Ltd., China, 215334.
- Qi Lv and Mike Zheng Shou are with the Department of Electrical and Computer Engineering and Show Lab, National University of Singapore, Singapore, 117583 (email: mike.zheng.shou@gmail.com).
- Mike Zheng Shou and Xiang Deng are corresponding authors.

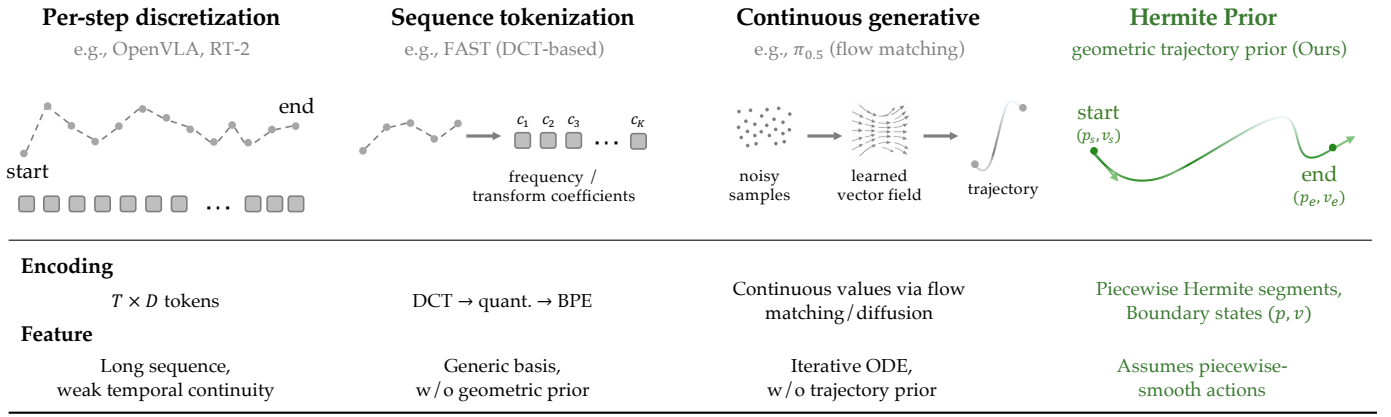


Fig. 1. Representations of an action chunk. Per-step discretization (e.g., OpenVLA, RT-2) emits one token per timestep, yielding long, weakly structured token sequences. Sequence tokenization (e.g., FAST) compresses the chunk into transform coefficients via a generic (DCT) basis with limited physical prior. Continuous generative decoders (e.g., $\pi_{0.5}$) sample a trajectory from a learned flow-matching field, expressive but geometry-agnostic. Our Hermite trajectory prior represents the chunk by endpoint positions and velocities (p, v) , making intra-chunk smoothness and inter-chunk continuity explicit in the action representation rather than leaving them to be learned implicitly from data.

enabling consecutive chunks to join smoothly during closed-loop execution. (2) It structurally guarantees smoothness for intra-chunk motion. By construction, each segment is C^1 -continuous, and the low-order polynomial basis naturally restricts representational capacity to low-frequency components. This eliminates high-frequency, jagged predictions that standard pointwise decoders often produce. In this way, temporal smoothness and boundary continuity, which standard decoders must attempt to learn implicitly from data, are structurally embedded within the action representation.

This prior is also lightweight and compatible with different action modeling paradigms. Reconstructing a dense trajectory from its boundary variables requires only a fixed basis-matrix multiplication, making the operator compact, differentiable, and inexpensive to integrate into existing VLA models. Using this common operator, we investigate two nested design questions. (1) *Can the same trajectory prior support both discrete and continuous action modeling?* We answer affirmatively by introducing two specialized realizations: Discrete Modeling with Hermite tokens (**HERMITE-VLA_{DH}**), which predicts quantized Hermite boundary parameters autoregressively, and Continuous Modeling with a Hermite Scaffold (**HERMITE-VLA_{CH}**), which decomposes each action estimate into a structural Hermite scaffold and a per-timestep residual. (2) *Within continuous action modeling, must the prior explicitly participate in deployed action prediction?* To explore this problem, we introduce Continuous Modeling with Hermite Regularization (**HERMITE-VLA_{Reg}**). This variant employs the Hermite operator strictly as an auxiliary training objective, preserving the exact baseline action head and sampling procedure during inference without introducing runtime latency.

We conduct comprehensive experiments across two simulation benchmarks (LIBERO and LIBERO-plus), along with three real-robot platforms spanning single-arm and bimanual manipulation. Empirical results demonstrate that Hermite trajectory priors consistently enhance performance across both discrete and continuous VLA paradigms. Against the shared π -series baseline, the strongest variant **HERMITE-VLA_{Reg}** improves the average success rate

from 95.9% to 98.7% on standard LIBERO, from 85.7% to 90.9% under the distribution shifts of LIBERO-plus, and from 63.4% to 90.0% on four real-robot tasks. Similarly, the discrete variant **HERMITE-VLA_{DH}** improves over its autoregressive baseline π_0 -FAST from 85.5% to 95.4% on LIBERO and from 61.6% to 69.4% on LIBERO-plus. Crucially, within continuous action modeling, **HERMITE-VLA_{Reg}** outperforms **HERMITE-VLA_{CH}** while retaining the baseline action head and sampling procedure at inference.

To summarize, our main contributions are as follows:

- Different from existing decoders that treat an action chunk as a flat sequence of per-timestep controls, we introduce a compact cubic Hermite operator that parameterizes action chunks through endpoint positions and velocities, thus embedding explicit intra-chunk smoothness and inter-chunk boundary continuity directly into VLA action learning.
- We systematically integrate this trajectory prior into both discrete and continuous VLA action modeling via **HERMITE-VLA_{DH}** and **HERMITE-VLA_{CH}**, while also introducing **HERMITE-VLA_{Reg}**, which enforces trajectory continuity purely through an auxiliary objective to preserve the original inference pipeline.
- Beyond empirical success rates, we provide a fine-grained kinematic characterization of the generated motions, demonstrating that the prior effectively lowers jerk, suppresses high-frequency spectral content, and attenuates seam transients on real hardware. Crucially, our findings reveal that trajectory smoothness benefits VLAs primarily as a learning-stage inductive bias rather than an inference-stage constraint, enabling the training-only **HERMITE-VLA_{Reg}** to yield optimal performance at zero runtime overhead.

2 RELATED WORK

2.1 Vision-Language-Action Models for Manipulation

Robot manipulation has long been studied through visuomotor imitation learning, where policies map visual

observations and robot states to low-level control commands [5], [6], [12], [13], [14], [15]. Recent Vision-Language-Action (VLA) models [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27] extend this paradigm by adapting large pretrained vision-language models to robot control, enabling policies to condition on visual observations and language instructions while benefiting from large-scale multimodal pretraining [1], [2], [3], [4], [28], [29], [30], [31]. This line of work has been further advanced by larger robot datasets [32], [33], [34], open-source training recipes [1], [30], and reasoning-oriented architectures [7], [35], [36], [37].

A central design choice in these systems is the action representation. Autoregressive VLA models discretize actions into tokens and emit them with a language-model head [1], [7], [29]. Continuous generative decoders model the action chunk as a conditional sample, using denoising diffusion [6], [30], [38], [39] or flow-matching objectives [2], [3]. These directions mainly differ in the functional form of the decoder and the scale of the pretraining data. Despite these differences, the models share a common assumption: the chunk is a flat matrix of per-timestep controls. They differ in *how* this action chunk is predicted, not in *what* representation of the action chunk. This paper instead revisits the representation itself, modeling the chunk as a continuous trajectory rather than a set of independent samples.

2.2 Action Chunk Representation

Action chunking predicts a short horizon of future controls rather than a single next action, and has become a widely adopted mechanism for reducing compounding error and improving temporal consistency in imitation learning [5], [6], [38], [40]. The standard representation treats a chunk as a matrix of raw continuous actions, such as end-effector poses, joint targets, or gripper commands [5], [6], [30]. Token-based policies instead quantize actions into per-dimension bins, making robot control compatible with sequence modeling and language-model decoding [1], [7], [29], [41]. Other work learns latent action codebooks [42], [43], [44] to better capture multimodal action distributions.

Recent work has also explored whole-chunk representations, as surveyed from an action-tokenization perspective in [10], [45], [46]. Most notably, FAST [47] applies a discrete cosine transform to the chunk and tokenizes the resulting spectral coefficients, reducing token length and improving the efficiency of autoregressive action decoding. These methods show that the representation of the action chunk is itself an important design choice, beyond the backbone and decoding objective. Hermite parameterization shares this representation-level view but differs in where the structure lives: spectral coefficients are global descriptors of the whole horizon and offer no handle on the states at chunk boundaries, whereas Hermite coordinates are the local boundary positions and velocities, aligning the representation with the replanning interface at which continuity is required to hold.

2.3 Trajectory Priors and Motion Primitives

More recently, classical motion parameterizations [48], [49], [50] have been revisited within learning-based policies. For instance, movement primitives have been integrated with

diffusion models to generate smooth manipulation trajectories [51], [52]. Concurrently with our work, Spline Policy [53] replaces standard action chunks with spline parameters across several backbones, directly using trajectory parameterizations as the policy output during both training and deployment. A separate line of work leaves the flat chunk intact and instead attempts to repair inter-chunk seams under asynchronous execution. These approaches condition newly predicted chunks on previously generated ones or correct already decoded outputs using external modules [9], [54], [55], [56], instead of explicitly modeling boundary states within the primary decoder. Moreover, beyond robotic manipulation, explicit temporal trajectory structures have proven similarly valuable in adjacent forecasting tasks, such as hand motion prediction [57], [58] and temporal action localization [59], [60].

In contrast to prior work, we systematically investigate the architectural level at which trajectory structure should be integrated into pretrained VLAs. We explore Hermite curves both as explicit policy output representations and as implicit trajectory priors. Crucially, our empirical analysis reveals that applying the Hermite structure strictly as an auxiliary training objective yields the highest performance. This implicit formulation achieves superior efficacy without incurring any inference-time overhead, unlocking a zero-cost deployment regime inaccessible to traditional primitive-as-policy architectures.

3 METHOD

We first describe the problem formulation and revisit the two typical modeling paradigms our variants build on (Section 3.1), then define the Hermite trajectory operator (Section 3.2), and finally instantiate it at three instantiations of VLA models (Section 3.3).

3.1 Preliminary

3.1.1 Problem Formulation

We cast robot manipulation as imitation learning from demonstrations. At each decision step the VLA model receives an observation $o = (\mathcal{I}, \ell, s)$ consisting of visual images $\mathcal{I}$, a textual instruction ℓ , and proprioceptive state s (e.g., joint positions). Following the common recipe, a pretrained vision-language backbone encodes the observation into a context representation c , and an action decoder conditioned on c predicts the robot action a .

Action chunking. To maintain the temporal coherence, modern VLAs adopt *action chunk* as the output, instead of the single step action. At each decision point the decoder predicts a chunk of T consecutive actions,

$$\mathbf{a} \in \mathbb{R}^{T \times D}, \quad (1)$$

where $\mathbf{a}$ is a sequence of T per-step controls, each of dimension D , typically comprising an end-effector pose together with a gripper command.

Closed-loop replanning. During deployment, the policy operates in a receding-horizon fashion by executing a predicted chunk, acquiring updated observations, and predicting a fresh chunk. Consecutive chunks are concatenated at

a *replanning seam* every W steps ($W \leq T$), where two independently decoded action sequences meet without explicit position or velocity continuity constraints. Although the decoder is trained via behavior cloning to reproduce target chunks, standard training objectives provide no guarantee of smoothness across replanning boundaries, resulting in abrupt velocity jumps, contact disruption, and task failure.

3.1.2 Discrete Autoregressive Modeling

Token-based VLAs [1], [29], [47] cast action prediction as language modeling. A fixed action tokenizer maps the chunk a to a sequence of discrete tokens $q_{1:L}$, and a language-model head factorizes their joint distribution autoregressively,

$$p_\phi(q_{1:L} | c) = \prod_{i=1}^L p_\phi(q_i | q_{<i}, c), \quad (2)$$

trained with a next-token cross-entropy loss. Per-dimension binning yields $L = TD$ tokens, whereas whole-chunk tokenizers such as FAST [47] transform the chunk before quantization to shorten L . At inference the tokens are decoded sequentially and detokenized back to a . This family reuses the language-model head unchanged, but both the achievable action granularity and the per-chunk decoding cost are dictated by the tokenizer.

3.1.3 Continuous Generative Modeling

Continuous modeling methods generate continuous a and thus avoid quantization. Flow matching action modeling [2], [3], [16], [30] models the multimodal action distribution by learning a time-dependent velocity field that transports Gaussian noise to data. We adopt the convention in which $t = 1$ is noise and $t = 0$ is the clean target, so that integrating from $t = 1$ to $t = 0$ denoises. Given a clean chunk $\mathbf{a}$ and noise $\epsilon \sim \mathcal{N}(0, I)$, the two are linearly interpolated, with target velocity

$$\mathbf{a}_t = t\epsilon + (1-t)\mathbf{a}, \quad u_t = \epsilon - \mathbf{a}. \quad (3)$$

The head $v_\phi(\mathbf{a}_t, t)$ is trained to match u_t [61], [62],

$$\mathcal{L}_{\text{flow}} = \mathbb{E}_{t, \epsilon} \|v_\phi(\mathbf{a}_t, t) - u_t\|_2^2, \quad (4)$$

and a chunk is sampled by integrating the field from $t = 1$ to $t = 0$ in N Euler steps of size $\Delta t = -1/N$,

$$\mathbf{a}_{t+\Delta t} = \mathbf{a}_t + \Delta t v_\phi(\mathbf{a}_t, t). \quad (5)$$

Since Eq. (3) is linear, predicting the velocity is equivalent to predicting the clean chunk $\hat{\mathbf{a}}_0$, through $\hat{\mathbf{a}}_0 = \mathbf{a}_t - t v_\phi(\mathbf{a}_t, t)$ and $v_t = (\mathbf{a}_t - \hat{\mathbf{a}}_0)/t$. The x_0 -prediction form, which estimates the clean chunk $\mathbf{a}$ at each step, exposes the chunk itself as the object on which a trajectory prior can act.

3.2 Action Chunks and Hermite Trajectory Operator

A VLA model emits a fixed-length action chunk $\mathbf{a} \in \mathbb{R}^{T \times D}$, where T denotes the number of future steps and D is the total action dimension. To account for distinct physical dynamics, we partition the action space and denote by $\mathbf{a}^c \in \mathbb{R}^{T \times D_c}$ the smooth continuous subspace of the action chunk, formed by selecting the $D_c \leq D$ continuously varying channels such as translational and rotational

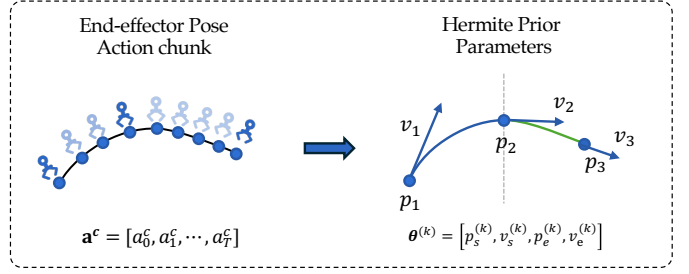


Fig. 2. Hermite encoding of an action chunk. For each segment k , the endpoint positions and velocities $\theta^{(k)} = (p_s^{(k)}, v_s^{(k)}, p_e^{(k)}, v_e^{(k)})$ are extracted from the end-effector chunk, yielding a compact trajectory representation.

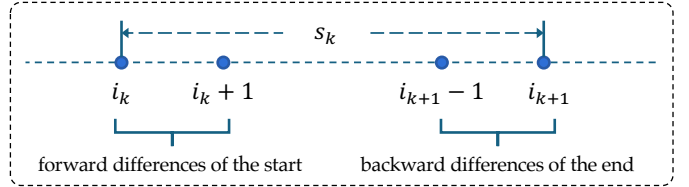


Fig. 3. One-sided finite-difference estimation of Hermite endpoint velocities. The start and end velocities of segment k are computed from forward and backward differences, respectively, and rescaled by the segment span s_k .

displacement components, while excluding discrete signals such as binary gripper commands.

The central object in our method is a fixed Hermite trajectory operator $\mathcal{D}_H(\theta) = \mathbf{H}\theta$, where $\theta \in \mathbb{R}^{4K \times D_c}$ contains the boundary variables of K cubic Hermite segments and $\mathbf{H} \in \mathbb{R}^{T \times 4K}$ is a precomputed basis matrix. The output $\mathcal{D}_H(\theta)$ is a dense trajectory over the horizon in the smooth subspace. Thus, θ serves as a structured latent variable for the smooth part of the action chunk, while $\mathcal{D}_H$ maps this latent variable back to per-timestep controls, the interface expected by existing VLA decoders.

As shown in Figure 2, an action chunk is first transformed to K segments. For each segment $k \in \{0, \dots, K-1\}$, the latent variable consists of endpoint positions and velocities,

$$\theta^{(k)} = (p_s^{(k)}, v_s^{(k)}, p_e^{(k)}, v_e^{(k)}) \in \mathbb{R}^{4D_c}, \quad (6)$$

where subscripts s and e denote the start and end of the segment, and p and v denote position and velocity, respectively. With local coordinate $u \in [0, 1]$, the segment is decoded as

$$a^{(k)}(u) = h_{00}(u)p_s^{(k)} + h_{10}(u)v_s^{(k)} + h_{01}(u)p_e^{(k)} + h_{11}(u)v_e^{(k)}, \quad (7)$$

where the cubic Hermite basis functions are

$$h_{00}(u) = 2u^3 - 3u^2 + 1, \quad h_{10}(u) = u^3 - 2u^2 + u, \\ h_{01}(u) = -2u^3 + 3u^2, \quad h_{11}(u) = u^3 - u^2. \quad (8)$$

Thus, we obtain the matrix $\mathbf{H}$ by tabulating these basis functions at the T control timestamps. Concretely, let timestamp i fall in segment $k(i)$ with local coordinate $u_i \in [0, 1]$. Row i of $\mathbf{H}$ is zero except for the four columns of segment $k(i)$,

Algorithm 1 Hermite Target Encoding and Trajectory Decoding (shared by all variants)

Require: chunk $\mathbf{a}_{\text{gt}} \in \mathbb{R}^{T \times D}$; segment count K ; precomputed basis $\mathbf{H} \in \mathbb{R}^{T \times 4K}$ (Eq. (9))

Ensure: boundary variables $\boldsymbol{\theta} \in \mathbb{R}^{4K \times D_c}$; decoded smooth chunk $\hat{\mathbf{a}}^c \in \mathbb{R}^{T \times D_c}$

```

1:  $\mathbf{a}_{\text{gt}}^c \leftarrow$  smooth channels of  $\mathbf{a}_{\text{gt}}$ 
2:  $i_k \leftarrow \text{round}(k(T-1)/K)$  for  $k = 0, \dots, K$ ;  $s_k \leftarrow i_{k+1} - i_k$ 
3: for  $k = 0, \dots, K-1$  do ▷ each segment encoded independently
4:    $p_s^{(k)} \leftarrow \mathbf{a}_{\text{gt}}^c[i_k]$ ;  $p_e^{(k)} \leftarrow \mathbf{a}_{\text{gt}}^c[i_{k+1}]$ 
5:    $v_s^{(k)} \leftarrow (\mathbf{a}_{\text{gt}}^c[i_{k+1}] - \mathbf{a}_{\text{gt}}^c[i_k]) s_k$  ▷ Eq. (10)
6:    $v_e^{(k)} \leftarrow (\mathbf{a}_{\text{gt}}^c[i_{k+1}] - \mathbf{a}_{\text{gt}}^c[i_{k+1}-1]) s_k$ 
7:    $\boldsymbol{\theta}^{(k)} \leftarrow (p_s^{(k)}, v_s^{(k)}, p_e^{(k)}, v_e^{(k)})$ 
8: end for
9: decode:  $\hat{\mathbf{a}}^c \leftarrow \mathcal{D}_H(\boldsymbol{\theta}) = \mathbf{H}\boldsymbol{\theta}$  ▷ one cached matrix multiply

```

namely columns $4k(i)$ through $4k(i)+3$, which hold the basis values:

$$\mathbf{H}_{i, 4k(i):4k(i)+3} = (h_{00}(u_i), h_{10}(u_i), h_{01}(u_i), h_{11}(u_i)), \quad (9)$$

giving $\mathbf{H}$ a block-banded structure that maps four boundary variables of each segment to the timestamps it spans. The overall procedure is summarized in Algorithm 1.

This construction satisfies three core requirements for a VLA-compatible trajectory prior. First, it establishes a low-dimensional structural scaffold for the smooth subspace of the action chunk, parameterizing motion with $4KD_c$ boundary variables rather than TD_c unconstrained samples. Second, since its parameterization consists directly of endpoint positions and velocities, boundary behavior becomes explicitly controllable at both interior segment knots and inter-chunk replanning seams. Third, since the operator $\mathcal{D}_H$ is a fixed linear mapping, trajectory-space losses propagate back to $\boldsymbol{\theta}$ analytically without specialized optimization procedures.

Target encoding. Given a ground-truth chunk $\mathbf{a}_{\text{gt}}$, we first extract its smooth subspace $\mathbf{a}_{\text{gt}}^c$. We then place $K+1$ knots at the rounded uniform grid $i_k = \text{round}(k(T-1)/K)$, so that adjacent segment spans $s_k = i_{k+1} - i_k$ may differ by one step. Each segment is encoded independently into its four boundary variables: the endpoint positions are read directly from the chunk, $p_s^{(k)} = \mathbf{a}_{\text{gt}}^c[i_k]$ and $p_e^{(k)} = \mathbf{a}_{\text{gt}}^c[i_{k+1}]$, and the endpoint velocities are the one-sided finite differences at the two ends as shown in Figure 3, rescaled by the segment span so that they are expressed in the local coordinate $u \in [0, 1]$,

$$\begin{aligned} v_s^{(k)} &= (\mathbf{a}_{\text{gt}}^c[i_k+1] - \mathbf{a}_{\text{gt}}^c[i_k]) s_k, \\ v_e^{(k)} &= (\mathbf{a}_{\text{gt}}^c[i_{k+1}] - \mathbf{a}_{\text{gt}}^c[i_{k+1}-1]) s_k. \end{aligned} \quad (10)$$

This deterministic encoder is shared across all variants, so differences among variants come from where the Hermite prior enters the VLA decoder rather than from different target construction. The encoded targets are C^0 -exact at interior knots, since the two copies of a shared knot read the same ground-truth frame. Tangent (C^1) behavior is soft:

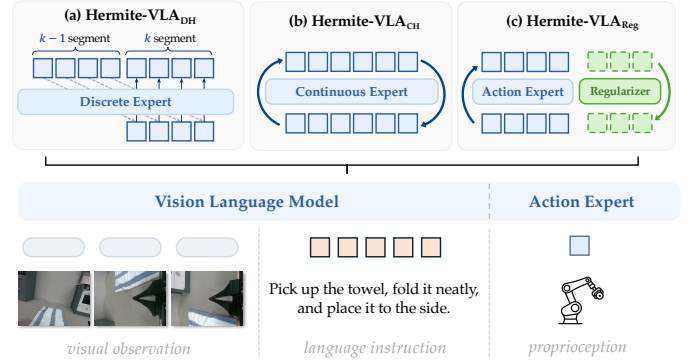


Fig. 4. Three instantiations of the Hermite trajectory prior. (a) Hermite Token (HERMITE-VLA_{DH}) tokenizes the boundary variables and decodes them segment by segment. (b) Hermite Scaffold (HERMITE-VLA_{CH}) decomposes each clean-action estimate into a Hermite scaffold and per-timestep residuals at every step of the flow matching ODE. (c) Hermite Regularization (HERMITE-VLA_{Reg}) attaches a training-only auxiliary Hermite head that shapes the shared features and is discarded at inference.

the two sides of an interior knot carry the backward and forward differences of that frame, whose gap is the local discrete curvature of the demonstration, so the encoding biases the decoder toward tangent-consistent motion without imposing it as a hard constraint.

3.3 Instantiating the Hermite Trajectory Prior

According to the action chunk modeling described in Sections 3.1.2 and 3.1.3, we introduce three variants and aim to explore two nested design questions. (1) *Can the same fixed Hermite operator be integrated into both discrete autoregressive and continuous generative action modeling?* To this end, HERMITE-VLA_{DH} reformulates discrete action modeling around quantized Hermite boundary variables, whereas HERMITE-VLA_{CH} incorporates a decoded Hermite scaffold into the clean-action prediction of a continuous generative model. (2) *Within continuous generative modeling, must the prior explicitly participate in deployed action prediction?* HERMITE-VLA_{Reg} provides an objective-only alternative that retains the original continuous action head and sampling procedure while using Hermite reconstruction as auxiliary supervision. Thus, HERMITE-VLA_{DH} and HERMITE-VLA_{CH} demonstrate its applicability across discrete and continuous action modeling, while HERMITE-VLA_{CH} and HERMITE-VLA_{Reg} compare an explicit Hermite decomposition of the deployed prediction against an objective-only use of the same operator. The training and inference procedures are summarized in Algorithms 2 and 3, respectively.

3.3.1 Discrete Modeling with Hermite Tokens

HERMITE-VLA_{DH} adapts the Hermite operator to discrete action modeling by treating its boundary variables as action tokens. An action chunk is first encoded into $\boldsymbol{\theta}$ by the shared target encoder. Each coordinate θ is then uniformly quantized into one of B bins over its respective training set range $[\theta_{\min}, \theta_{\max}]$:

$$q(\theta) = \text{round}\left(\frac{\text{clip}(\theta, \theta_{\min}, \theta_{\max}) - \theta_{\min}}{\theta_{\max} - \theta_{\min}}(B-1)\right). \quad (11)$$

Algorithm 2 Training of the Three Hermite-VLA Variants

Require: observation o ; demonstration chunk $\mathbf{a}_{\text{gt}}$; basis $\mathbf{H}$; auxiliary weight λ

Shared computation

- 1: $\mathbf{c} \leftarrow \text{VLM}(o)$ $\triangleright$ backbone evaluated once
- 2: $\boldsymbol{\theta}_{\text{gt}} \leftarrow \text{HERMITEENCODE}(\mathbf{a}_{\text{gt}})$ $\triangleright$ Alg. 1
- 3: compute $\mathcal{L}$ by the variant's block below; update parameters by $\nabla \mathcal{L}$

① Hermite-VLA_{DH} (segment-level autoregressive)

- 4: $\mathbf{q} \leftarrow$ quantize $\boldsymbol{\theta}_{\text{gt}}$ into B bins $\triangleright$ Eq. (11)
- 5: $\mathcal{L} \leftarrow \mathcal{L}_{\text{DH}}(\mathbf{q}; \mathbf{c}) +$ gripper cross-entropy $\triangleright$ Eq. (12)

② Hermite-VLA_{CH} (scaffold + residual)

- 6: $t \sim \mathcal{U}(0, 1), \epsilon \sim \mathcal{N}(0, \mathbf{I}); \mathbf{a}_t \leftarrow t\epsilon + (1-t)\mathbf{a}_{\text{gt}}$ $\triangleright$ Eq. (3)
- 7: $\mathbf{h} \leftarrow \text{ActionExpert}(\mathbf{a}_t, t, \mathbf{c}); \boldsymbol{\theta} \leftarrow \text{MLP}\left(\frac{1}{T} \sum_{\tau} \mathbf{h}_{\tau}\right); \mathbf{r}_{\tau} \leftarrow \mathbf{W}_r \mathbf{h}_{\tau}$ $\triangleright$ Eq. (16)
- 8: $\mathcal{L} \leftarrow \|\mathbf{H}\boldsymbol{\theta} + \mathbf{r} - \mathbf{a}_{\text{gt}}\|_2^2 +$ non-smooth-channel loss $\triangleright$ Eq. (17)

③ Hermite-VLA_{Reg} (auxiliary objective only)

- 9: $t, \epsilon, \mathbf{a}_t, \mathbf{h}$ as in lines 6–7
- 10: $\mathcal{L}_{\text{flow}} \leftarrow \|v_{\phi}(\mathbf{a}_t, t) - (\epsilon - \mathbf{a}_{\text{gt}})\|_2^2; \boldsymbol{\theta}_{\text{aux}} \leftarrow \text{MLP}_{\text{aux}}\left(\frac{1}{T} \sum_{\tau} \mathbf{h}_{\tau}\right)$ $\triangleright$ Eq. (4); no residual
- 11: $\mathcal{L} \leftarrow \mathcal{L}_{\text{flow}} + \lambda \|\mathbf{H}\boldsymbol{\theta}_{\text{aux}} - \mathbf{a}_{\text{gt}}\|_2^2$ $\triangleright$ Eqs. (19) and (20)

Algorithm 3 Inference of the Three Hermite-VLA Variants

Require: context $\mathbf{c} = \text{VLM}(o)$; basis $\mathbf{H}$; denoising steps N

① Hermite-VLA_{DH} (segment-level autoregressive)

- 1: **for** $k = 0, \dots, K-1$ **do**
- 2: decode token group $\hat{\mathbf{q}}^{(k)}$ conditioned on $\hat{\mathbf{q}}^{(<k)}, \mathbf{c}$ $\triangleright 4D_c$ tokens in parallel
- 3: **end for**
- 4: $\hat{\boldsymbol{\theta}} \leftarrow$ dequantize $\hat{\mathbf{q}}$ $\triangleright$ Eq. (13)
- 5: $\hat{\mathbf{a}}^c \leftarrow \mathbf{H}\hat{\boldsymbol{\theta}}$; append gripper-head output

②/③ Hermite-VLA_{CH} and Hermite-VLA_{Reg} (flow matching)

- 6: $\mathbf{a} \leftarrow \epsilon \sim \mathcal{N}(0, \mathbf{I})$
 - 7: **for** $n = 1, \dots, N$ **do** $\triangleright$ Euler steps, $\Delta t = -1/N$
 - 8: $\hat{\mathbf{a}}_0 \leftarrow \text{CLEANESTIMATE}(\mathbf{a}, t_n, \mathbf{c})$
 - 9: $\mathbf{a} \leftarrow \mathbf{a} + \Delta t (\mathbf{a} - \hat{\mathbf{a}}_0)/t_n$ $\triangleright$ Eq. (18)
 - 10: **end for**
- CLEANESTIMATE: scaffold-plus-residual $\mathbf{H}\boldsymbol{\theta} + \mathbf{r}$ concatenated with the non-smooth channels for CH; the *unchanged* baseline head for REG (auxiliary branch discarded)

This yields $4KD_c$ tokens per chunk, organized into K groups of $4D_c$ tokens, one group for each Hermite segment. We denote the quantized token group of segment k by $\mathbf{q}^{(k)}$.

Considering the causal relationship between segments, we adopt *segment-level autoregressive* prediction. Specifically, the vision-language backbone obtains context $\mathbf{c}$ by encoding the observation o , and then a lightweight action head predicts the segment groups conditioned on $\mathbf{c}$ sequentially. Within each segment, the $4D_c$ coordinate tokens are emitted in parallel:

$$\mathcal{L}_{\text{DH}} = - \sum_{k=0}^{K-1} \sum_{j=1}^{4D_c} \log p_{\phi}(q_j^{(k)} | \mathbf{q}^{(<k)}, \mathbf{c}). \quad (12)$$

Thus, each segment conditions on all preceding segment groups, while the tokens within the same segment share a common segment context and do not condition on one another. The gripper channel is predicted independently at each timestep by a separate output head.

During training, the logits for all segment groups can be computed in one forward pass under the teacher forcing setting. At inference, the segment groups are decoded sequentially. After each group is predicted, its token embeddings are supplied as context for the next segment. Let $\hat{q} \in \{0, \dots, B-1\}$ denotes a predicted bin index, and the continuous boundary value $\hat{\theta}$ is then dequantized according to

$$\hat{\theta} = \frac{\hat{q}}{B-1} (\theta_{\max} - \theta_{\min}) + \theta_{\min}, \quad (13)$$

and assembled into the predicted boundary variables $\hat{\boldsymbol{\theta}}$. The smooth action trajectory is reconstructed as

$$\hat{\mathbf{a}}^c = \mathcal{D}_H(\hat{\boldsymbol{\theta}}). \quad (14)$$

The separately predicted gripper channel is combined with the $\hat{\mathbf{a}}^c$ to form the complete action chunk $\hat{\mathbf{a}}$.

Unlike existing action tokenizers such as FAST [47], HERMITE-VLA_{DH} assigns each token an explicit trajectory-level meaning: the endpoint position or velocity of a Hermite segment. Moreover, its autoregressive loop is confined to the K segment groups of a lightweight action head, without inserting the action tokens into the backbone's autoregressive sequence, making the inference more efficient. However, discretizing these continuous boundary variables inevitably introduces quantization error, motivating the continuous variants introduced next.

3.3.2 Continuous Modeling with a Hermite Scaffold

HERMITE-VLA_{CH} adapts the Hermite operator to continuous generative action modeling under the same x_0 -prediction formulation as the baseline $\pi_{0.5}$ [3]. At each flow time t , the action head decomposes the clean estimate of the smooth action channels into a Hermite scaffold and a per-timestep residual:

$$\hat{\mathbf{a}}_0^c = \underbrace{\mathcal{D}_H(\boldsymbol{\theta})}_{\text{Hermite scaffold}} + \underbrace{\mathbf{r}}_{\text{per-timestep residual}} = \mathbf{H}\boldsymbol{\theta} + \mathbf{r}. \quad (15)$$

The residual compensates for high-frequency micro-adjustments and complex local variations that fall outside the expressive range of the low-dimensional Hermite basis. Formally, let $\mathbf{h} \in \mathbb{R}^{T \times d}$ denote the features produced by the action expert for the noised chunk $\mathbf{a}_t$. The Hermite branch aggregates features over the full horizon, whereas the residual branch produces a separate correction for each timestep:

$$\boldsymbol{\theta} = \text{MLP}\left(\frac{1}{T} \sum_{\tau=1}^T \mathbf{h}_{\tau}\right), \quad \mathbf{r}_{\tau} = \mathbf{W}_r \mathbf{h}_{\tau}. \quad (16)$$

This separates a low-dimensional, chunk-level trajectory component from time-indexed residual corrections.

We adopt a plain x_0 reconstruction training objective on the clean action trajectory:

$$\mathcal{L}_{\text{CH}} = \mathbb{E}_{t, \epsilon} \left[\|\mathbf{H}\boldsymbol{\theta} + \mathbf{r} - \mathbf{a}_{\text{gt}}^c\|_2^2 \right]. \quad (17)$$

Notably, we apply supervision in trajectory space rather than directly to $\boldsymbol{\theta}$. The finite-difference target encoder provides one deterministic estimate of the boundary velocities, whereas trajectory-space supervision requires only that the reconstructed action trajectory match the demonstration.

At inference, the smooth estimate in Eq. (15) is combined with the separately predicted non-smooth channels to form the complete action estimate $\hat{\mathbf{a}}_0(\mathbf{a}_t, t, \mathbf{c})$. The corresponding velocity $\mathbf{v}_t$ and Euler update are

$$\mathbf{v}_t = \frac{\mathbf{a}_t - \hat{\mathbf{a}}_0(\mathbf{a}_t, t, \mathbf{c})}{t}, \quad \mathbf{a}_{t+\Delta t} = \mathbf{a}_t + \Delta t \mathbf{v}_t. \quad (18)$$

Thus, the Hermite prior actively conditions the generative trajectory during every solver step rather than being applied as an external post-processing operation after sampling.

3.3.3 Continuous Modeling with Hermite Regularization

While HERMITE-VLA_{CH} explicitly incorporates the Hermite scaffold into every action prediction, a trajectory prior need not necessarily alter the deployed action parameterization. It serves as a training-time structural objective that encourages the shared action features to capture the global trajectory underlying a dense sequence of per-timestep controls. Accordingly, HERMITE-VLA_{Reg} introduces a low-dimensional Hermite reconstruction task alongside the original action-prediction objective, while leaving the deployed action head and flow-matching sampler unchanged. Since the auxiliary branch summarizes the full horizon through endpoint positions and velocities, it provides explicit boundary-aware supervision that is absent from the standard dense action prediction objective.

In parallel with the main action head, an auxiliary Hermite branch predicts $\boldsymbol{\theta}$ from the shared features. It adopts the same pooled architecture as the Hermite branch in Eq. (16), but omits the per-timestep residual. The predicted boundary variables are decoded through $\mathcal{D}_H$ and supervised against the clean smooth-action trajectory:

$$\mathcal{L}_{\text{herm}} = \mathbb{E}_{t, \epsilon} \left[\|\mathbf{H}\boldsymbol{\theta} - \mathbf{a}_{\text{gt}}^c\|_2^2 \right]. \quad (19)$$

The complete training objective is

$$\mathcal{L}_{\text{Reg}} = \mathcal{L}_{\text{flow}} + \lambda \mathcal{L}_{\text{herm}}, \quad (20)$$

where λ controls the relative weight of the auxiliary Hermite reconstruction. Since the auxiliary head is used exclusively during training and removed at inference, HERMITE-VLA_{Reg} preserves the original action head, sampling procedure, and inference efficiency without incurring any computational overhead.

Although the auxiliary prediction is computed from features conditioned on the noised chunk $\mathbf{a}_t$, its target is always the clean trajectory $\mathbf{a}_{\text{gt}}^c$. The same trajectory-level target is therefore presented across flow times and noise realizations, encouraging the shared features to retain a consistent description of the underlying motion. This distinguishes HERMITE-VLA_{Reg} from HERMITE-VLA_{CH}: HERMITE-VLA_{CH} uses the Hermite scaffold as an explicit component of the deployed clean-action estimate, whereas HERMITE-VLA_{Reg} uses the same operator only to enrich the learning signal. As a result, HERMITE-VLA_{Reg} introduces boundary-aware trajectory supervision without quantization, modifying the runtime output parameterization, or adding inference overhead.

3.4 Theoretical Analysis of the Hermite Prior

We introduce three elementary analyses that make precise why the Hermite coordinates are a suitable interface for VLA action learning. Statements are given for one smoothly varying channel, i.e., $\boldsymbol{\theta} \in \mathbb{R}^{4K}$ and $\mathbf{a}^c \in \mathbb{R}^T$, and extend to the D_c channels column-wise. Throughout we assume that every segment contains at least four sampled timestamps (Appendix A, satisfied at $K=2$), which guarantees that $\mathbf{H}$ has full column rank with smallest singular value $\sigma_{\min}(\mathbf{H}) > 0$. Each property accounts for one empirical finding of Section 4: the supervision-space ablation (Table 7), the bell-shaped dependence on K (Table 6), and the seam reduction (Table 12).

Proposition 3.1 (Supervision geometry). Let $P_{\mathbf{H}} = \mathbf{H}(\mathbf{H}^\top \mathbf{H})^{-1} \mathbf{H}^\top$ and, for a target $\mathbf{a} \in \mathbb{R}^T$, let $\boldsymbol{\theta}_{\text{LS}}(\mathbf{a}) = (\mathbf{H}^\top \mathbf{H})^{-1} \mathbf{H}^\top \mathbf{a}$. For every $\boldsymbol{\theta} \in \mathbb{R}^{4K}$,

$$\|\mathbf{H}\boldsymbol{\theta} - \mathbf{a}\|_2^2 = \|\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{LS}}(\mathbf{a})\|_{\mathbf{H}^\top \mathbf{H}}^2 + \|(\mathbf{I} - P_{\mathbf{H}}) \mathbf{a}\|_2^2, \quad (21)$$

where $\|\mathbf{x}\|_M^2 = \mathbf{x}^\top M \mathbf{x}$: the trajectory-space losses of Eqs. (17) and (19) supervise the same least-squares target as a θ -space loss, but under the metric $\mathbf{H}^\top \mathbf{H}$, which weights each boundary coordinate by exactly its influence on the executed trajectory.

The two metrics exhibit a substantial disparity in scale: the position columns of $\mathbf{H}$ possess elements of order $O(1)$, whereas the velocity columns are strictly bounded by $\max_u |h_{10}| = \max_u |h_{11}| = 4/27$. Consequently, an identity-metric loss in θ -space disproportionately penalizes velocity errors relative to their actual impact on the trajectory footprint. Reformulating the objective within trajectory space directly mitigates this weighting imbalance.

Proposition 3.2 (Expressivity and bandwidth of the scaffold). Suppose the smooth channel samples an underlying trajectory, $\mathbf{a}_{\text{gt}}^c[i] = f(i)$ with $f \in C^2([0, T-1])$, and let $\boldsymbol{\theta}_{\text{gt}}$ be produced by the target encoder of Eq. (10). Then, (i) $\mathcal{D}_H(\boldsymbol{\theta}_{\text{gt}})$ is exact at every knot and $\|\mathcal{D}_H(\boldsymbol{\theta}_{\text{gt}}) - \mathbf{a}_{\text{gt}}^c\|_\infty \leq \frac{3}{8} s_{\max}^2 \|f''\|_\infty$, where $s_{\max} = \max_k s_k \approx (T-1)/K$ and time is measured in control steps; (ii) every trajectory in the span of $\mathcal{D}_H$ has piecewise-linear acceleration and piecewise-constant jerk taking at most K distinct values, bounded on segment k by

$$\left| \frac{d^3 \mathbf{a}^{(k)}}{dt^3} \right| \leq \frac{12 |p_s^{(k)} - p_e^{(k)}|}{s_k^3} + \frac{6 (|\nu_s^{(k)}| + |\nu_e^{(k)}|)}{s_k^2}, \quad (22)$$

where $\nu^{(k)} = v^{(k)}/s_k$ are the per-step endpoint velocities.

The two bounds pull in opposite directions: the approximation error of the target scaffold falls as K^{-2} while the admissible jerk ceiling grows as K^3 , so K trades the low-frequency bias of the prior for fidelity, predicting the interior optimum observed at $K=2$. Part (i) also shows that the residual $\mathbf{r}$ of Eq. (15) only needs to model a second-order remainder and any content outside C^2 . For HERMITE-VLA_{Reg}, both parts describe the auxiliary *target*, i.e., the function class toward which the shared features are regularized, rather than the deployed policy.

Proposition 3.3 (Seam control). For any execution window $W \leq T$, the incoming side of a handover is exactly the

TABLE 1

Real-robot tasks and demonstration data. All demonstrations are collected by human teleoperation under end-effector control at 30 Hz. “#Objects” denotes the number of manipulated objects per rollout (randomized where a range is given), and “2-Wrist” indicates two wrist-mounted cameras.

Task	Platform	Instruction	Teleoperation	#Objects	#Demos	Duration	Views
1	Franka (single-arm)	Open the pot, put the wooden block into it, and close the pot.	GELLO [63]	1–3	832	6.3 h	Side/1-Wrist
2	Cybopal (single-arm)	Open the pot, put the wooden block into it, and close the pot.	PICO VR	1–3	1,006	4.1 h	Top/1-Wrist
3	Cybopal (dual-arm)	Left arm opens the pot lid, right arm puts the blocks and balls into the pot, and closes the lid.	PICO VR	1–3	805	4.6 h	2-Wrist/Front
4	ARX (dual-arm)	Pick up the towel, fold it neatly, and place it to the side.	PICO VR	1	723	3.2 h	2-Wrist/Front

boundary-coordinate pair $(\hat{p}_s^{(0)}, \hat{v}_s^{(0)})$ of the new chunk, and the outgoing side is a fixed linear functional $(\mathbf{r}_p^\top \hat{\boldsymbol{\theta}}, \mathbf{r}_v^\top \hat{\boldsymbol{\theta}})$ of the previous chunk, given by the rows of $\mathbf{H}$ and of its derivative counterpart at the last executed timestep. When this timestep falls on a segment knot (in particular under full-chunk execution, $W = T$), the outgoing state reduces to boundary coordinates as well. Every such error is bounded through $\|\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}_{\text{LS}}(\mathbf{a})\|_2 \leq \|\mathbf{H}\hat{\boldsymbol{\theta}} - \mathbf{a}\|_2 / \sigma_{\min}(\mathbf{H})$, up to an absolute factor below 1.03 in the functional case (Appendix A). Consequently, the seam discontinuity in position and velocity is bounded by the prediction errors of the two adjacent chunks together with the across-seam increment intrinsic to the demonstration, and is therefore directly controlled by the trajectory-space objectives of Eqs. (12), (17) and (19).

Overall, the distinction from standard flat representations is what the parameters expose: in a flat chunk the seam *position* is directly a coordinate, but the *velocity* of the handover transition is a finite difference bridging consecutive chunks; it is a function of neither individual parameter set and thus goes unmonitored under standard per-chunk behavior cloning objectives. Two caveats qualify the analysis. First, the bound accumulates only per-chunk errors, so the prior narrows seam discontinuities rather than eliminating them. Second, the guarantee concerns the Hermite scaffold, i.e., before residual addition in HERMITE-VLA_{CH}, and enters HERMITE-VLA_{Reg} only as an inductive bias imparted through shared features rather than as a hard runtime constraint.

4 EXPERIMENTS

4.1 Experimental Setup

Simulation Benchmarks. We adopt two simulation benchmarks. *Standard LIBERO* [64] consists of four task suites, namely Spatial, Object, Goal, and 10, each containing 10 tasks. *LIBERO-plus* [65] augments these four suites with seven categories of test-time perturbation, namely Camera, Robot, Language, Light, Background, Noise, and Layout, and after filtering and category balancing yields 10,030 perturbed task instances that probe robustness to distribution shift while leaving the nominal goal unchanged.

Real-world platforms. We further evaluate on four real-robot manipulation tasks across three hardware platforms, as shown in Figure 5, spanning single-arm and dual-arm settings. Table 1 summarizes the tasks, instructions, and demonstration data. Each task is defined as a three-stage

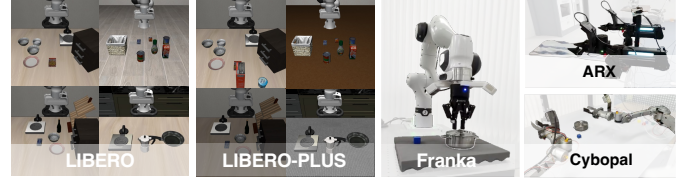


Fig. 5. Evaluation platforms. Two simulation benchmarks, LIBERO and LIBERO-plus, and three real-robot platforms: a single-arm Franka (7 DoF), a Cybopal arm (6 DoF per arm) used in both single-arm and dual-arm configurations, and a dual-arm ARX platform (6 DoF per arm).

sequence (open $\rightarrow$ place $\rightarrow$ close for Tasks 1–3, pick $\rightarrow$ fold $\rightarrow$ place for Task 4) so that partial progress can be measured. The four tasks are designed to disentangle task difficulty from embodiment: Tasks 1 and 2 share the same instruction on different single-arm platforms, Task 3 moves to a dual-arm configuration of the same platform for coordinated placement, and Task 4 probes deformable-object manipulation on a separate dual-arm platform.

Evaluation protocol. On standard LIBERO we follow the official protocol and evaluate each task over 50 rollouts with randomized initial states, giving 500 episodes per suite, and report the per-suite success rate (SR) and the four-suite average. On LIBERO-plus, we report the per-category SR together with the average over all 10,030 instances. On the real robot, each task is evaluated through 15 runs with random placement of object positions and random number of objects. A rollout is counted as successful only if the final stage is completed, and we additionally report a cumulative stage-wise pass rate, defined as the fraction of the three stages a rollout completes in order.

Trajectory-quality metrics. While task success rates measure macro-level completion, they fail to capture fine-grained execution dynamics. We therefore quantify three trajectory-level metrics, targeting the intra-chunk roughness and inter-chunk discontinuity:

- **Jerk** (time-domain smoothness): Computed as the third time-derivative of position along trajectories. Lower magnitudes signify attenuated mechanical vibrations, reduced tracking errors, and enhanced control stability.
- **SPARC** (frequency-domain smoothness): Evaluated via the Spectral Arc Length of the velocity profile in frequency. Less negative values indicate suppressed high-frequency spectral content, suppressing execution jitter.
- **Seam Discontinuity** (inter-chunk boundary consis-

TABLE 2

Standard LIBERO success rate (%) on the four official suites and their average. Params denotes the approximate model size. The top block lists autoregressive token-prediction policies and the bottom block continuous action-generation policies. The best average within each block is in bold.

Method	Params	Training Objective	LIBERO-Spatial	LIBERO-Object	LIBERO-Goal	LIBERO-10	Avg.
<i>Autoregressive token-prediction policies</i>							
OpenVLA [1]	~7B	AR CE	84.7	88.4	79.2	53.7	76.5
WorldVLA [66]	~7B	AR CE	87.6	96.2	83.4	60.0	81.8
π_0 -FAST [47]	~3.3B	AR CE	96.4	96.8	88.6	60.2	85.5
Hermite-VLA _{DH} (Ours)	~3.3B	AR CE	96.0	98.6	96.4	90.4	95.4
<i>Continuous action-generation policies</i>							
Diffusion Policy [6]	157M	Diffusion	78.3	92.5	68.3	50.5	72.4
Octo [30]	93M	Diffusion	78.9	85.7	84.6	51.1	75.1
DiT Policy [67]	334M	Diffusion	84.2	96.3	85.4	63.8	82.4
OpenVLA-OFT [68]	~7B	L1 regression	97.6	98.4	97.9	94.5	97.1
$\pi_{0.5}$ [3]	~3.3B	Flow matching	98.4	96.0	97.8	91.4	95.9
Hermite-VLA _{CH} (Ours)	~3.3B	Flow matching	99.2	99.2	96.4	96.0	97.7
Hermite-VLA _{Reg} (Ours)	~3.3B	Flow matching + Hermite reg.	99.6	99.2	99.0	96.8	98.7

tency): Quantified by the step-displacement ratio at chunk handovers relative to nominal execution per step. Lower values denote seamless closed-loop replanning transitions.

Baselines. We compare against representative policies from both decoder families and group the results accordingly. Among autoregressive token-prediction policies we include OpenVLA [1], WorldVLA [66], and π_0 -FAST [47], whose FAST tokenizer is the direct counterpart of HERMITE-VLA_{DH} on the same backbone family. Among continuous action-generation policies we include Diffusion Policy [6], Octo [30], DiT Policy [67], OpenVLA-OFT [68], and the flow-matching $\pi_{0.5}$ [3], which is also the backbone shared by all three Hermite-VLA variants. For each baseline we report numbers from its original publication where the evaluation protocol matches ours, and otherwise reproduce it according to its original training setting.

4.2 Implementation Details

Backbones. For our models, all three variants are instantiated on π -series [3] backbone, a PaliGemma vision-language tower [69] with an action expert. HERMITE-VLA_{DH} replaces the π_0 -FAST tokenizer with the hermite tokenizer; HERMITE-VLA_{CH} restructures the clean-action prediction of the flow-matching head into the scaffold-plus-residual form; HERMITE-VLA_{Reg} attaches the auxiliary head and leaves the deployed head unchanged. All runs (the baselines π_0 -FAST, $\pi_{0.5}$, HERMITE-VLA_{DH}, HERMITE-VLA_{CH}, and HERMITE-VLA_{Reg}) share one training framework and the same training setting, while differ only in the head and the loss, so their comparison is controlled.

Trajectory operator. All three variants use $K = 2$ segments on the rounded uniform grid. On LIBERO and LIBERO-plus, $T = 10$, $W = 5$, $D = 7$, and $D_c = 6$, excluding the binary gripper channel. On the real-robot platforms, $T = 50$, $W = 20$ at a 30 Hz control rate, with end-effector control throughout: $D = 7$, $D_c = 6$ for the single-arm Franka and Cybopal setups, and $D = 14$, $D_c = 12$ for the dual-arm Cybopal and ARX setups, excluding one binary gripper channel per arm. The basis matrix $\mathbf{H}$ is precomputed once and cached on the accelerator. The non-smooth channels are predicted by the separate tail and concatenated

with the decoded smooth trajectory. The auxiliary and replacement Hermite heads use a two-layer MLP of hidden width 512, and HERMITE-VLA_{DH} quantizes each Hermite coordinate into 256 bins.

Training parameters. All variants are initialized from the open-source π -series (the discrete variant from π_0 -FAST and continuous variants from $\pi_{0.5}$) base checkpoint, with the newly introduced Hermite or auxiliary heads randomly initialized, and fine-tuned with AdamW under a cosine learning-rate schedule warming up over 1k steps to a peak of 5×10^{-5} and decaying to 3×10^{-5} , batch size 256, and quantile action normalization. We adopt the $\lambda = 10$ for HERMITE-VLA_{Reg} training and all variants train for 30k steps with $8 \times$ H100 GPUs. At inference, the continuous variants sample with $N=10$ denoising steps.

4.3 Main Results

4.3.1 Simulation Results

Standard LIBERO. We summarize the quantitative performance across all four LIBERO suites in Table 2. For autoregressive policies, HERMITE-VLA_{DH} achieves a 95.4% average success rate, substantially surpassing π_0 -FAST (85.5%). For continuous policies, HERMITE-VLA_{Reg} and HERMITE-VLA_{CH} attain 98.7% and 97.7% average success rates respectively, both outperforming the $\pi_{0.5}$ baseline (95.9%). The gains are most substantial on the challenging LIBERO-10 suite, where HERMITE-VLA_{Reg} elevates the baseline from 91.4% to 96.8%. HERMITE-VLA_{DH} follows the same trend, gaining 30.2% (60.2% $\rightarrow$ 90.4%) over π_0 -FAST on this suite. These results strongly align with our analysis in Section 1. Long-horizon tasks in LIBERO-10 involve frequent replanning seams over fixed execution windows. While flat representations provide no trajectory-level mechanisms to regularize motion across these chunk boundaries, Hermite variants expose boundary states via structured tokens, scaffold decomposition, or auxiliary regularization. This substantial boost confirms that explicit boundary parameterization effectively prevents error accumulation during extended closed-loop execution.

LIBERO-plus. Table 3 reports per-category and average success rates under the seven environmental perturbations.

TABLE 3

LIBERO-plus success rate (%) under the seven test-time perturbation categories, together with the instance-weighted average (Avg.) over all 10,030 instances. Params denotes the approximate model size. The top block lists autoregressive token-prediction policies and the bottom block continuous action-generation policies. The best result in each column within each block is in bold.

Method	Params	Camera	Robot	Language	Light	Background	Noise	Layout	Instance Avg.
<i>Autoregressive token-prediction policies</i>									
OpenVLA [1]	~7B	0.8	3.5	23.0	8.1	34.8	15.2	28.5	15.6
WorldVLA [66]	~7B	0.1	27.9	41.6	43.7	19.8	10.9	38.0	25.3
π_0 -FAST [47]	~3.3B	65.1	21.6	61.0	73.2	73.2	74.4	68.8	61.6
Hermite-VLA _{DH} (Ours)	~3.3B	54.3	36.9	70.4	92.8	86.4	78.8	77.6	69.4
<i>Continuous action-generation policies</i>									
OpenVLA-OFT [68]	~7B	56.4	31.9	79.5	88.7	93.3	75.8	74.2	69.6
$\pi_{0.5}$ [3]	~3.3B	75.8	79.4	83.3	95.5	95.0	89.6	87.0	85.7
Hermite-VLA _{CH} (Ours)	~3.3B	78.7	83.3	97.7	87.0	85.9	73.9	86.8	85.0
Hermite-VLA _{Reg} (Ours)	~3.3B	89.2	85.4	87.3	97.7	96.7	93.8	89.9	90.9

TABLE 4

Real-robot success rate (%) on four manipulation tasks, 15 rollouts each with randomized object placement. The cumulative stage-wise pass rate indicates *where* rollouts fail: S1 and S2 denote completion of the first and second stages, and SR denotes overall success (completion of the final stage). The best SR per task is in bold.

Task	Method	S1	S2	SR
Task 1	π_0 -FAST	93.3	60.0	60.0
	$\pi_{0.5}$	100.0	93.3	86.7
	Hermite-VLA _{DH}	100.0	80.0	80.0
	Hermite-VLA _{CH}	100.0	86.7	86.7
	Hermite-VLA _{Reg}	100.0	100.0	100.0
Task 2	π_0 -FAST	53.3	20.0	20.0
	$\pi_{0.5}$	86.7	26.7	26.7
	Hermite-VLA _{DH}	66.7	33.3	33.3
	Hermite-VLA _{CH}	93.3	60.0	60.0
	Hermite-VLA _{Reg}	80.0	66.7	66.7
Task 3	π_0 -FAST	86.7	46.7	40.0
	$\pi_{0.5}$	93.3	53.3	46.7
	Hermite-VLA _{DH}	93.3	46.7	46.7
	Hermite-VLA _{CH}	100.0	80.0	80.0
	Hermite-VLA _{Reg}	100.0	93.3	93.3
Task 4	π_0 -FAST	86.7	86.7	80.0
	$\pi_{0.5}$	93.3	93.3	93.3
	Hermite-VLA _{DH}	93.3	93.3	86.7
	Hermite-VLA _{CH}	100.0	100.0	100.0
	Hermite-VLA _{Reg}	100.0	100.0	100.0
Avg	π_0 -FAST	80.0	53.4	50.0
	$\pi_{0.5}$	93.3	66.7	63.4
	Hermite-VLA _{DH}	88.3	60.8	61.7
	Hermite-VLA _{CH}	98.3	81.7	81.7
	Hermite-VLA _{Reg}	95.0	90.0	90.0

Among continuous policies, HERMITE-VLA_{Reg} achieves the best overall average of 90.9% and ranks first in six of the seven categories, improving over the $\pi_{0.5}$ baseline (85.7%) in every category. Its largest gains occur under Camera and Robot perturbations, by 13.4% and 6.0%, respectively, whereas the improvements are smaller for appearance-only shifts such as Light (+2.2%) and Background (+1.7%). This profile is consistent with an action-side prior that primarily structures how motion is generated rather than how visual appearance is represented. HERMITE-VLA_{CH} reaches 85.0% average SR, leading on geometric perturbations but

falling below the baseline on Light, Background, and Noise. The comparison suggests that, when the pretrained action predictor already models dense actions effectively, Hermite structure is more beneficial as complementary supervision. HERMITE-VLA_{Reg} preserves the original action head while adding an auxiliary trajectory objective, yielding more consistent gains across perturbations. Among autoregressive policies, HERMITE-VLA_{DH} reaches 69.4% average SR, compared with 61.6% for π_0 -FAST, showing that the prior also benefits discrete action modeling.

4.3.2 Real-world Results

Table 4 reports per-task success on the four real-robot tasks. Among continuous policies, HERMITE-VLA_{Reg} reaches the best average SR at 90.0%, against 63.4% for the $\pi_{0.5}$ baseline and 81.7% for HERMITE-VLA_{CH}. The gain is largest on two tasks, Task 2 (26.7% $\rightarrow$ 66.7%) and Task 3 (46.7% $\rightarrow$ 93.3%); Task 1 rises to 100% and Task 4, already near ceiling, stays there. The autoregressive pair preserves its simulation ordering on hardware, HERMITE-VLA_{DH} clearly ahead of π_0 -FAST, though both trail the continuous methods. The stage-wise breakdown locates where the difference arises: the first stage (opening the lid or picking up the object) is solved at or near 100% by all methods (except π_0 -FAST), whereas the final SR is governed almost entirely by the middle stage, the precise placement or fold, whose pass rate on Task 2 and Task 3 rises from 26.7% and 53.3% under the baseline to 66.7% and 93.3% under HERMITE-VLA_{Reg}. On Task 2 this improvement holds even though the first-stage rate is lower than the baseline, indicating that the gain comes from the execution of the manipulation itself rather than from reaching the manipulation more reliably. We relate this to the executed motion in Section 4.5.

4.4 Ablation Studies

Injection strength (λ). To evaluate the sensitivity of HERMITE-VLA_{Reg} to the auxiliary regularization weight λ , we conduct an ablation scan on standard LIBERO as shown in Table 5. At $\lambda=0$, the objective reverts to the unregularized baseline. As λ increases, the average success rate exhibits a characteristic bell-shaped profile, peaking at 98.7% at $\lambda=10$ before slightly declining to 97.5% at $\lambda=20$. This slight drop indicates that an overly aggressive

TABLE 5

Injection-strength sensitivity for HERMITE-VLA_{Reg} on standard LIBERO across auxiliary-loss weights λ (at $K = 2$). The best average success rate and the lowest median jerk are in bold.

λ	Spatial	Object	Goal	10	Avg.	Median Jerk $\downarrow$
0	98.4	96.0	97.8	91.4	95.9	0.000460
1	98.6	98.8	97.4	94.8	97.4	0.000443
5	99.8	99.8	98.4	94.6	98.2	0.000422
10	99.6	99.2	99.0	96.8	98.7	0.000410
20	99.0	99.2	97.2	94.6	97.5	0.000418

TABLE 6

Segment-count ablation for HERMITE-VLA_{Reg} on standard LIBERO as the number of Hermite segments K is varied (at $\lambda = 10$). The best average success rate and the lowest median jerk are in bold.

K	Spatial	Object	Goal	10	Avg.	Median Jerk $\downarrow$
1	96.2	94.0	96.4	88.0	93.7	0.000451
2	99.6	99.2	99.0	96.8	98.7	0.000410
3	97.0	95.6	98.2	95.4	96.6	0.000426
4	96.0	98.2	97.8	91.0	95.8	0.000439

auxiliary loss competes with the primary flow-matching objective. Crucially, as λ scales from 0 to 10, the median execution jerk drops monotonically from 0.00046 to 0.00041, confirming that trajectory-level regularization directly enhances physical motion smoothness. While performance on shorter-horizon suites saturates early above 97%, the long-horizon LIBERO-10 suite proves most sensitive to regularization, surging from 91.4% to 96.8% (+5.4% absolute gain). This improvement underscores the vital role of boundary-aware trajectory priors in extended closed-loop manipulation tasks.

Segment count (K). We evaluate the sensitivity of HERMITE-VLA_{Reg} to the number of Hermite segments K at a fixed weight of $\lambda=10$. As shown in Table 6, the four-suite average success rate exhibits a clear bell-shaped trend that peaks at $K=2$ (98.7%). Under-parameterization ($K=1$) restricts expressiveness, making it insufficient to capture complex multi-stage motion chunks (93.7%). Conversely, over-parameterization ($K=3, 4$) increases representation capacity but degrades success rates to 96.6% and 95.8%, respectively. This trend is further corroborated by physical execution metrics, as $K=2$ achieves the lowest median jerk, whereas higher segment counts increase trajectory jitter. This performance degradation is most pronounced on the long-horizon LIBERO-10 benchmark, where $K=2$ outperforms $K=1$ by a substantial +8.8% margin. Given a fixed horizon of $T=10$, allocating excessive knot points reduces each segment span to only a few discrete timesteps, tightly coupling boundary velocities to adjacent frames and diluting the low-frequency smoothness prior.

Continuous-head design choices. To ablate the individual effects of Hermite scaffold and residual components, we systematically evaluate supervision representations and target configurations for HERMITE-VLA_{CH} in Table 7. Within θ -space, supervising boundary variables against global least-squares targets (Row 1) underperforms the local finite-difference encoder (Row 2) by 4.9% (91.5% vs. 96.4%). This drop indicates that global least-squares optimization dif-

TABLE 7

Supervision-design ablation for HERMITE-VLA_{CH} on standard LIBERO. Rows 1–2 apply supervision directly in θ -space under least-squares and finite-difference targets, respectively; Rows 3–4 supervise the reconstructed trajectory, where Row 4 additionally incorporates per-timestep residuals (the full HERMITE-VLA_{CH}). The best average is in bold.

Supervision	Target	Res.	Spatial	Object	Goal	10	Avg.
θ -space	least-squares	$\times$	94.6	92.6	94.4	85.0	91.5
θ -space	finite-diff	$\times$	98.2	97.2	96.2	91.6	96.4
Trajectory	(implicit LS)	$\times$	98.0	98.2	95.4	93.2	96.2
Trajectory	–	$\checkmark$	99.2	99.2	96.4	96.0	97.7

TABLE 8

Boundary-basis ablation on standard LIBERO at $K = 2$. The best average success rate and the lowest median jerk are in bold.

Basis	Spatial	Object	Goal	10	Avg.	Median Jerk $\downarrow$
Polynomial	98.6	97.8	97.8	92.4	96.7	0.000447
Bernstein	98.0	97.0	98.8	93.8	96.9	0.000439
B-spline	98.2	98.8	98.4	95.0	97.6	0.000422
Ours	99.6	99.2	99.0	96.8	98.7	0.000410

fuses fitting errors across the entire horizon, weakening the local physical grounding of boundary parameters. Shifting supervision directly to trajectory space without residuals (Row 3) implicitly recovers the least-squares objective while effectively re-weighting the optimization metric through the normal matrix $\mathbf{H}^\top \mathbf{H}$. The resulting 4.7% performance surge (91.5% to 96.2%) demonstrates that the failure in Row 1 stems from unweighted coordinate representation in θ -space rather than the target formulation itself. Notably, trajectory-space supervision matches the performance of the best engineered θ -space target while eliminating manual target encoding, and even outperforms it on the long-horizon LIBERO-10 suite (93.2% vs. 91.6%). Finally, incorporating per-timestep residuals (Row 4) provides a clean single-factor boost to 97.7%, confirming that while the Hermite scaffold imposes an essential structural prior, per-timestep residuals are necessary to capture high-frequency details outside the cubic spline span.

Boundary basis. To verify whether the performance gain stems from the specific coordinate parameterization rather than general polynomial trajectory smoothing, we compare the cubic Hermite operator against three standard alternatives under identical evaluation settings [70], [71]: a global cubic polynomial, a per-segment Bernstein basis, and a uniform cubic B-spline. While all candidates define fixed linear mappings into low-order polynomial trajectory spaces, the key distinction lies in the coordinate representation driven by the action head. Hermite parameters represent boundary positions and velocities directly, yielding physically meaningful quantities that naturally align with action normalization statistics. In contrast, Bernstein control points relate to boundary kinematics only indirectly through linear basis transformations, whereas global polynomial and B-spline parameterizations impose structural constraints that restrict expressiveness over the trajectory family. As reported in Table 8, our Hermite representation achieves the highest average success rate (98.7%) and the lowest execution jerk (0.000410). This advantage highlights an optimization-level

TABLE 9

Inference cost on a single NVIDIA H100 80 GB GPU, averaged over 1,000 end-to-end action-chunk generations from a fixed LIBERO observation (two camera views).

Variant	Latency (ms) ↓	FPS ↑	Peak Mem. (GB) ↓
$\pi_{0.5}$ -FAST	238.2	4.2	7.57
Hermite-VLA _{DH}	28.9	34.6	6.66
$\pi_{0.5}$	48.4	20.7	6.63
Hermite-VLA _{CH}	49.4	20.2	6.64
Hermite-VLA _{Reg}	48.6	20.6	6.85

alignment achieved by explicitly mapping latent coordinates to seam-governing physical states, an effect most pronounced on the long-horizon LIBERO-10 benchmark where boundary continuity dictates success.

Inference cost. To verify that introducing structured trajectory priors does not compromise real-time control efficiency, we evaluate the mean per-chunk generation latency and peak GPU memory on a single NVIDIA H100 GPU in Table 9. Across the flow-matching variants, execution overhead remains virtually indistinguishable: baseline $\pi_{0.5}$ (48.4 ms), HERMITE-VLA_{Reg} (48.6 ms), and HERMITE-VLA_{CH} (49.4 ms) all operate within a narrow latency window while maintaining a tight peak memory footprint between 6.63 and 6.85 GB. This empirical result validates the zero-cost deployment property established in Section 3.3.3, confirming that stripping the auxiliary Hermite head at inference introduces no operational penalty. For autoregressive generation, HERMITE-VLA_{DH} achieves the lowest latency (28.9 ms/34.6 FPS) by executing the transformer backbone only once and subsequently decoding compact trajectory tokens via an internal $K=2$ loop without ODE integration. In contrast, the token-by-token $\pi_{0.5}$ -FAST baseline suffers substantial latency (238.2 ms) due to repeated backbone forward passes for each individual action token. Overall, these benchmarks confirm that the proposed Hermite prior imposes no computational or memory overhead at deployment across both continuous and discrete VLA architectures.

4.5 Smoothness and Trajectory-Quality Analysis

Two policies may attain comparable task success rates while exhibiting qualitatively distinct execution kinematics. Therefore, we investigate two fundamental complementary questions: does the Hermite prior systematically refine the geometric *profile* of executed motion, and does this refinement yield tangible deployment advantages? To address this, we evaluate end-effector trajectory traces across both simulated LIBERO rollouts and real-world physical executions. We explicitly restrict this comparative evaluation to the continuous policy family ($\pi_{0.5}$, HERMITE-VLA_{CH}, HERMITE-VLA_{Reg}). By evaluating policies that operate within the same continuous flow-matching sampling framework, this controlled setup isolates motion smoothness as a function of the Hermite injection scheme, while avoiding confounding noise introduced by discrete tokenization and autoregressive decoding dynamics.

Global smoothness. To quantitatively assess holistic trajectory quality across long execution horizons, we evaluate end-effector translational speed trajectories using the

TABLE 10

Spectral Arc Length (SPARC) of end-effector translational speed on LIBERO, averaged over 500 rollouts per cell. Less negative is smoother.

Variant	Spatial	Object	Goal	10	Avg.
$\pi_{0.5}$	−2.293	−2.307	−2.396	−2.733	−2.432
HERMITE-VLA _{CH}	−2.261	−2.257	−2.336	−2.717	−2.403
HERMITE-VLA _{Reg}	−2.235	−2.220	−2.313	−2.658	−2.356

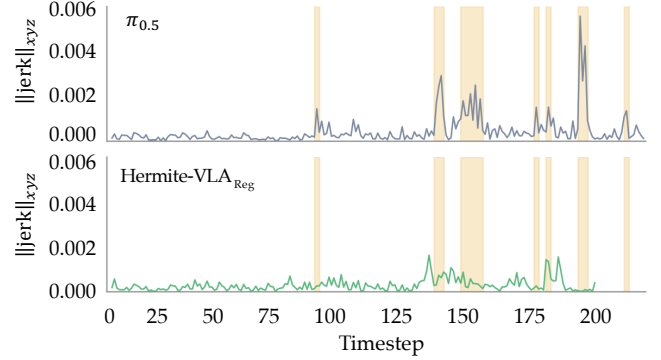


Fig. 6. Per-step jerk magnitude $\|jerk_{xyz}(t)\|$ along one LIBERO-10 episode that $\pi_{0.5}$ (top) and HERMITE-VLA_{Reg} (bottom) both complete from the same initial state. Shaded bands mark the jerk spikes of $\pi_{0.5}$, reproduced at the same positions in both panels.

SPARC metric across 2,000 LIBERO rollouts (Table 10). Less negative SPARC values indicate higher spectral smoothness. Across all benchmark suites, HERMITE-VLA_{Reg} consistently achieves the highest overall smoothness, attaining a four suite average of -2.356 compared to -2.403 for HERMITE-VLA_{CH} and -2.432 for $\pi_{0.5}$. The strict hierarchy (HERMITE-VLA_{Reg} > HERMITE-VLA_{CH} > $\pi_{0.5}$) holds universally across all four benchmark suites, confirming that Hermite regularization provides consistent spectral smoothness gains from short horizon manipulation to extended task execution.

Temporal localization of motion roughness. Aggregate metrics can obscure *when* motion instability occurs during execution. To resolve this temporal dimension, Figure 6 traces the per-step end-effector jerk magnitude across a representative LIBERO-10 episode, evaluated on both $\pi_{0.5}$ and HERMITE-VLA_{Reg} under identical initial conditions. The jerk profile of $\pi_{0.5}$ reveals sustained control instability during task execution, marked by an elevated noise floor and sharp transient spikes that reach nearly 0.006. As highlighted by the shaded intervals, these regions mark the periods where $\pi_{0.5}$ experiences severe motion bursts. Across these exact same shaded regions, HERMITE-VLA_{Reg} suppresses both the baseline signal noise and the transient discontinuities, maintaining peak jerk spikes below 0.002 while reaching task completion in fewer execution steps. This temporal trace explains the global gains in Table 10: Hermite prior reduces overall jerk by eliminating sharp, localized motion spikes, rather than simply dampening an already smooth trajectory across the board.

Smoothness tracks task outcome. To verify that the observed smoothness gains are not merely an artifact of higher success rates, such as survivor bias, we disentangle trajectory quality from task outcome. As shown in Table 11, we

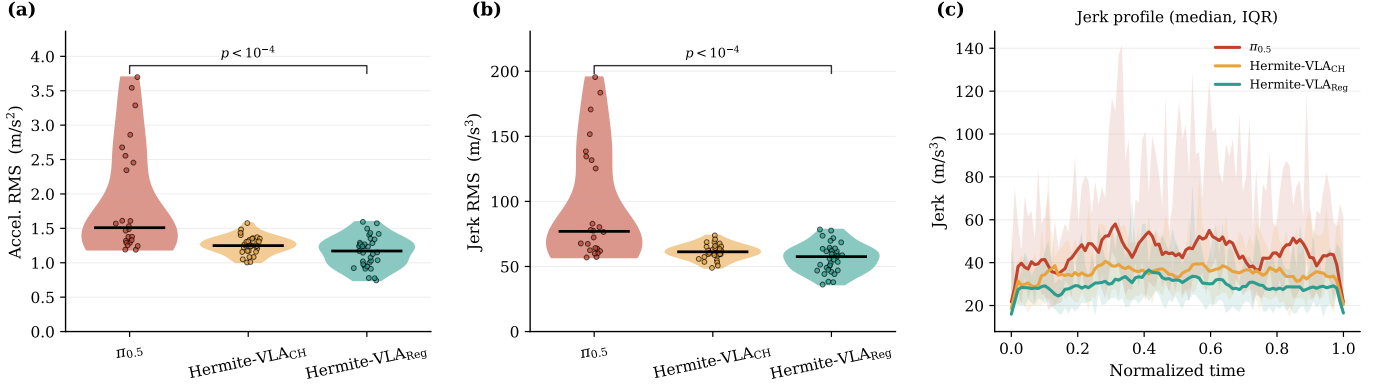


Fig. 7. Episode-level smoothness on the real robot. (a) Acceleration RMS and (b) jerk RMS per episode. Both Hermite variants are significantly smoother than $\pi_{0.5}$ ($p < 10^{-4}$). (c) Median jerk over normalized episode time with inter-quartile band: $\pi_{0.5}$ is the highest with the widest spread, HERMITE-VLA_{Reg} the lowest and most consistent.

TABLE 11

Trajectory smoothness versus task outcome on LIBERO. For each variant, the mean jerk magnitude $\|\text{jerk}_{xyz}\|$ is reported over successful versus failed rollouts, together with the number of failed episodes. Failed rollouts are markedly less smooth than successful ones across all variants; HERMITE-VLA_{Reg} is the smoothest on both subsets and fails least.

Variant	jerk (success)	jerk (failure)	# failed
$\pi_{0.5}$	0.00046	0.00070	82
Hermite-VLA _{CH}	0.00047	0.00052	46
Hermite-VLA _{Reg}	0.00040	0.00048	26

divide all rollouts into successful and failed execution subsets and find that failed episodes exhibit consistently higher jerk across all policy variants. Crucially, HERMITE-VLA_{Reg} achieves the lowest jerk within both the successful (0.00040) and failed (0.00048) subsets, confirming that its kinematic smooth execution holds independently of task outcome. While this tight correlation does not fully resolve causal directionality [72], [73], [74], namely whether physical motion roughness causes manipulation failures or out of distribution states trigger erratic control, both perspectives converge on a common insight. Specifically, HERMITE-VLA_{Reg} achieves the highest smoothness across both subsets while incurring the lowest failure count (26 failed episodes, compared to 82 for $\pi_{0.5}$ and 46 for HERMITE-VLA_{CH}). This indicates that Hermite regularization simultaneously eliminates physical control instability and prevents policy drift away from the training distribution. Consequently, improved task success and enhanced trajectory smoothness are not independent findings, but joint manifestations of a fundamentally more robust control policy.

Seam discontinuity on real-world platforms. To directly quantify physical control discontinuities at replanning boundaries and complement our holistic hardware kinematic analysis, we evaluate acceleration transients at replanning handovers across precision placement tasks (Table 12). Since velocity discontinuities at handovers manifest as sudden acceleration spikes, we measure the seam magnitude $\|a_t - a_{t-1}\|$ at every handover step and compare it against interior trajectory steps. We summarize three main insights

TABLE 12

Real-robot seam discontinuity ($\Delta a_{\text{seam}} = \|a_t - a_{t-1}\|$) on commanded end-effector translation at replanning handovers ($W=20$, $T=50$), reported as median (IQR). “rel.” is the seam median relative to the $\pi_{0.5}$ baseline, and $\rho = \Delta a_{\text{seam}} / \Delta a_{\text{interior}}$ measures the discontinuity ratio of handover steps to interior trajectory steps.

Task	Variant	$\Delta a_{\text{seam}} (\Delta a_{\text{interior}})$	rel.	ρ
Task 2	$\pi_{0.5}$	0.0056 (0.0036)	1.00×	7.6
	HERMITE-VLA _{CH}	0.0041 (0.0035)	0.73×	6.3
	HERMITE-VLA _{Reg}	0.0040 (0.0033)	0.72×	7.0
Task 3	$\pi_{0.5}$	0.0059 (0.0076)	1.00×	8.6
	HERMITE-VLA _{CH}	0.0034 (0.0040)	0.58×	5.7
	HERMITE-VLA _{Reg}	0.0028 (0.0027)	0.48×	5.5

from this boundary evaluation. First, every policy variant exhibits significantly higher control discontinuity at replanning handovers than during interior execution (ρ ranging from 5.5 to 8.6), which is consistent with the observation in simulation analysis. Second, Hermite regularization substantially mitigates this handover jump, with HERMITE-VLA_{Reg} reducing the relative seam median to 0.72× on Task 2 and 0.48× on Task 3 compared to $\pi_{0.5}$ baseline. Since Hermite regularization simultaneously smooths interior motion, the ratio ρ conservatively understates these absolute boundary gains, paving the way for the holistic kinematic smoothness achieved across the entire rollout. Third, while Hermite regularization narrows the seam spike, it does not completely eliminate it, which aligns with its formulation as a soft prior whose decoder is trained against boundary quantities governing handover transitions.

Kinematic gains transfer to hardware. Beyond overall task success, we specifically analyze the kinematic smoothness of real robot rollouts to evaluate whether Hermite regularization reduces physical joint wear and mechanical vibration on hardware as follows.

(1) To quantify overall motion quality and its temporal persistence during execution, we evaluate the time domain acceleration and jerk metrics across all rollouts. As shown in Figure 7(a) and Figure 7(b), both Hermite variants achieve significantly lower acceleration RMS and jerk RMS than $\pi_{0.5}$ ($p < 10^{-4}$, two sided Mann Whitney U test), with HERMITE-

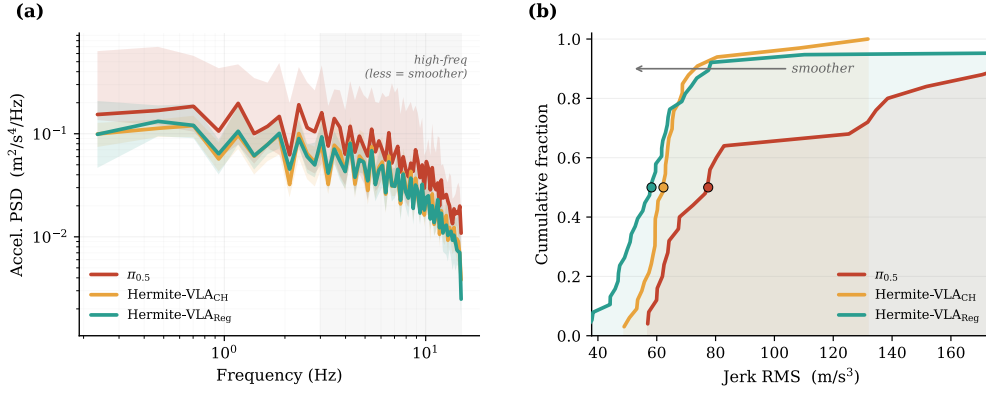


Fig. 8. Real-robot frequency-domain smoothness. (a) Welch acceleration power spectral density; high-frequency content manifests as jitter on hardware. (b) Cumulative distribution of per-episode jerk RMS.

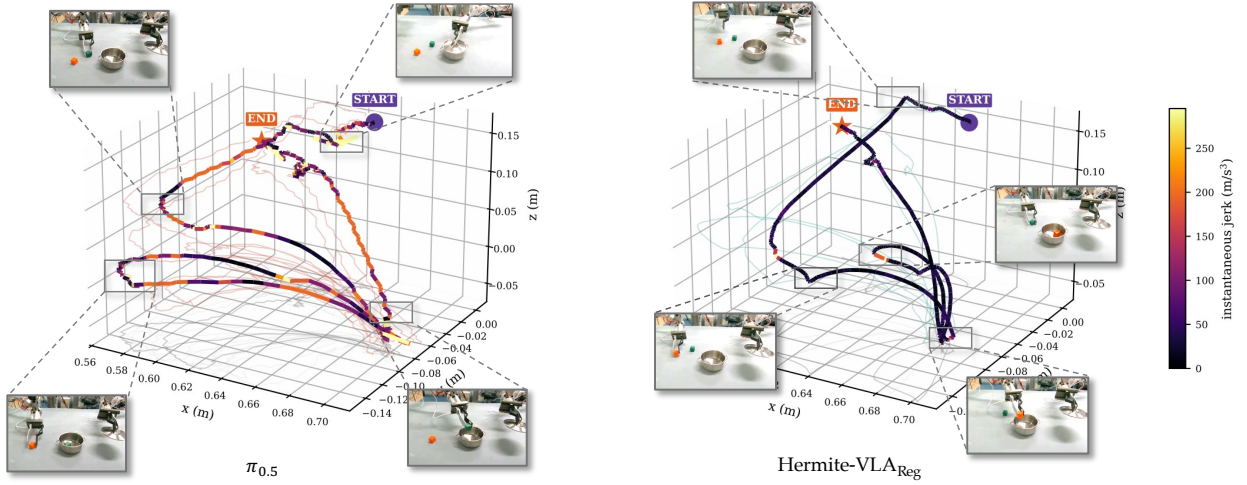


Fig. 9. Executed end-effector paths on Task 3 for $\pi_{0.5}$ (left) and $\text{HERMITE-VLA}_{\text{Reg}}$ (right) in the same scene. Thin lines show all rollouts. The bold line is one representative rollout, colored by instantaneous jerk (m/s^3).

VLA_{Reg} recording the lowest median jerk RMS. This advantage holds throughout execution: the time normalized jerk profiles in Figure 7(c) show that baseline $\pi_{0.5}$ maintains an elevated median jerk baseline, whereas Hermite variants suppress jerk magnitudes across the entire rollout.

(2) To uncover how control smoothness reflects spectral energy distribution, we examine the frequency characteristics of the executed motions. As depicted in Figure 8(a), frequency analysis confirms that Hermite regularization attenuates high frequency acceleration power spectral density (PSD), significantly eliminating rapid oscillations that induce physical jitter. Consequently, as shown in Figure 8(b), the cumulative distribution of jerk RMS shifts substantially toward lower values across rollouts.

(3) To intuitively understand how these kinematic gains translate to spatial execution quality, we map Cartesian end-effector pose trajectories across all successful rollouts with instantaneous jerk values. While $\pi_{0.5}$ exhibits severe jerk spikes exceeding 200 m/s^3 along representative rollouts and substantial path dispersion across all trials (Figure 9), $\text{HERMITE-VLA}_{\text{Reg}}$ consistently traces clean, low jerk 3D trajectories toward identical targets.

Qualitative Trajectory Analysis. To intuitively analyze how trajectory smoothness affects physical execution and

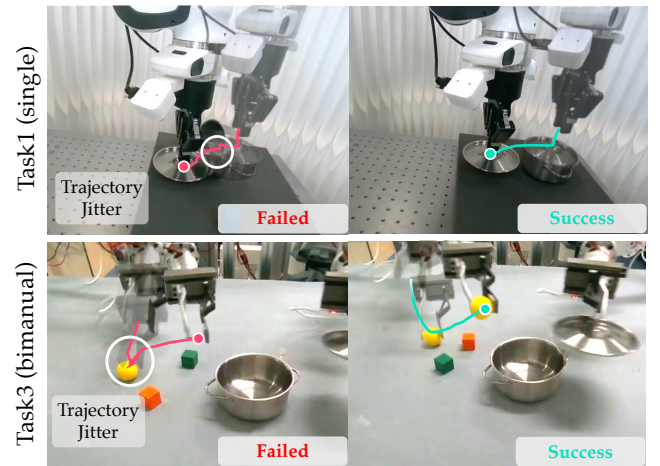


Fig. 10. Effect of trajectory regularity on real-robot tasks. $\pi_{0.5}$ (left) exhibits end-effector jitter near the object, perturbing the gripper-object alignment and resulting in failed grasps in both single-arm and bimanual tasks. $\text{HERMITE-VLA}_{\text{Reg}}$ (right) produces a smoother approach trajectory, maintains stable alignment before contact and move successfully.



Fig. 11. Qualitative comparison of end-effector trajectories in simulation. The $\pi_{0.5}$ baseline exhibits local oscillations, abrupt corrections, and longer paths during object approach and placement, as highlighted by the numbered regions. Hermite Regularization produces smoother and more direct trajectories and reaches successful task completion earlier.

task completion efficiency, we visualize representative end-effector trajectories on real-world platforms (Figure 10) and in simulation (Figure 11).

On real-world platforms, these kinematic defects directly determine execution success or failure. As shown in Figure 10, $\pi_{0.5}$ baseline suffers from severe trajectory jitter immediately before object engagement in both single arm and bimanual precision tasks. This sudden jitter disrupts gripper alignment, resulting in missed grasps and physical task failure. By enforcing Hermite regularization, the policy maintains a smooth, continuous approach that preserves precise alignment through contact, directly converting trajectory regularity into reliable physical success.

For the simulation, detailed temporal traces reveal how $\pi_{0.5}$ suffers from pervasive local oscillations, detours, and execution delays. As illustrated in Figure 11, during initial object approach (①), $\pi_{0.5}$ generates jagged zigzag waypoints, whereas HERMITE-VLA_{Reg} follows a direct, continuous path. This discrepancy compounds over extended execution (②): by $t = 102$, HERMITE-VLA_{Reg} has already transported the object directly over the container, whereas $\pi_{0.5}$ baseline lags substantially behind due to cumulative trajectory detours. Furthermore, during final object release (③, ④), $\pi_{0.5}$ executes redundant looping re corrections before completion. In contrast, HERMITE-VLA_{Reg} eliminates path redundancy, completing the lower task by $t = 88$, which is twice as fast as $\pi_{0.5}$ ($t = 176$).

5 LIMITATION AND FUTURE WORK

Our study highlights two key methodological trade-offs. First, Hermite regularization models trajectory smoothness as a soft inductive bias rather than strict constraints, preserving policy expressivity at the expense of absolute trajectory continuity. Second, the proposed prior focuses on action-sequence structure, operating independently of vision-side representations and of non-smooth contact events. Future research will explore tighter couplings between trajectory priors and visual perception, task adaptive chunk segmentation, and extensions toward highly dynamic, contact rich manipulation tasks.

6 CONCLUSION

In this paper, we comprehensively revisit the representation of action chunks, demonstrating that they are far more effectively modeled as continuous trajectories rather than flat sequences of discrete controls. To this end, we introduce Hermite trajectory priors, utilizing piecewise cubic Hermite segments to explicitly capture the underlying endpoint positions and velocities that govern intra chunk smoothness and inter chunk continuity. Built upon a single differentiable operator, this prior is instantiated at three distinct integration levels: HERMITE-VLA_{DH} tokenizes boundary variables for segment level autoregressive decoding, HERMITE-VLA_{CH} decomposes clean action estimates into a Hermite scaffold with per timestep residuals, and HERMITE-VLA_{Reg} applies

the operator purely as an auxiliary training objective. Across LIBERO, LIBERO plus, and four real world manipulation tasks, incorporating Hermite priors consistently boosts execution success across both discrete and continuous VLA architectures. Notably, the training only HERMITE-VLA_{Reg} variant delivers the largest empirical gains at zero inference cost while producing measurably smoother motion and more reliable physical rollouts. These findings demonstrate that trajectory structure serves as a powerful, lightweight inductive bias without disrupting pretrained representations, offering a promising paradigm for future VLA design.

REFERENCES

- [1] M. J. Kim, K. Pertsch *et al.*, “Openvla: An open-source vision-language-action model,” in *Proceedings of The 8th Conference on Robot Learning (CoRL)*, vol. 270, 2025, pp. 2679–2713.
- [2] K. Black, N. Brown, D. Driess *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [3] Physical Intelligence *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” in *Proceedings of The 9th Conference on Robot Learning (CoRL)*, vol. 305, 2025, pp. 17–40.
- [4] S. Liu, L. Wu, B. Li *et al.*, “RDT-1B: a diffusion foundation model for bimanual manipulation,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [5] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [6] C. Chi, S. Feng, Y. Du *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [7] Q. Li, Y. Liang, Z. Wang, L. Luo *et al.*, “CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation,” *arXiv preprint arXiv:2411.19650*, 2024.
- [8] L. Zhong, Y. Liu, Y. Wei, Z. Xiong, S. Liu, and G. Ren, “Acotvla: Action chain-of-thought for vision-language-action models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026, pp. 8152–8162.
- [9] K. Black, M. Y. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [10] H. Zhou, W. Liao *et al.*, “BEAST: Efficient tokenization of b-splines encoded action sequences for imitation learning,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [11] Z. Yang, Y. Shi, M. Yao, W. Xue, Y. Jueluo, and L. Liu, “Chunk-flow: Towards continuity-consistent chunked policy learning,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026.
- [12] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-end training of deep visuomotor policies,” *Journal of Machine Learning Research (JMLR)*, vol. 17, no. 1, pp. 1334–1373, 2016.
- [13] D. Wang, C. Liu, F. Chang, and Y. Xu, “Hierarchical diffusion policy: Manipulation trajectory generation via contact guidance,” *IEEE Transactions on Robotics (T-RO)*, vol. 41, pp. 2086–2104, 2025.
- [14] C. Finn and S. Levine, “Deep visual foresight for planning robot motion,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [15] T. Zhang, Z. McCarthy *et al.*, “Deep imitation learning for complex manipulation tasks from virtual reality teleoperation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [16] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da *et al.*, “Gr00t n1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [17] Gemini Robotics Team, “Gemini robotics: Bringing ai into the physical world,” *arXiv preprint arXiv:2503.20020*, 2025.
- [18] C.-L. Cheang, G. Chen, Y. Jing *et al.*, “Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation,” *arXiv preprint arXiv:2410.06158*, 2024.
- [19] M. Shukor, D. Aubakirova, F. Capuano *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [20] Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li, “UniVLA: Learning to act anywhere with task-centric latent actions,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [21] W. Zhang, H. Liu *et al.*, “Dreamvla: A vision-language-action model dreamed with comprehensive world knowledge,” 2025.
- [22] W. Wu, F. Lu *et al.*, “A pragmatic vla foundation model,” 2026.
- [23] X. R. Team, “Xiaomi-robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories,” 2026.
- [24] H. Luo, Y. Wang *et al.*, “Being-h0.5: Scaling human-centric robot learning for cross-embodiment generalization,” 2026.
- [25] H. Yuan, Z. Liang *et al.*, “Qwen-robotmanip technical report: Alignment unlocks scale for robotic manipulation foundation models,” 2026.
- [26] J. Sun, W. Zhang *et al.*, “Vla-jepa: Enhancing vision-language-action model with latent world model,” 2026.
- [27] W. Kong, Z. Su, W. Yu, and H. Dong, “Affordvla: Injecting affordance representations into vision-language-action models via implicit feature alignment,” 2026.
- [28] A. Brohan, N. Brown, J. Carbajal *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Robotics: Science and Systems (RSS)*, 2023.
- [29] —, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning (CoRL)*, 2023.
- [30] Octo Model Team, D. Ghosh, H. Walke *et al.*, “Octo: An open-source generalist robot policy,” in *Robotics: Science and Systems (RSS)*, 2024.
- [31] H. Li, G. Zhao *et al.*, “Ace-ego-0: Unifying egocentric human and robotic data for vla pretraining,” 2026.
- [32] A. Khazatsky, K. Pertsch, S. Nair *et al.*, “DROID: A large-scale in-the-wild robot manipulation dataset,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [33] H. Walke, K. Black, A. Lee *et al.*, “BridgeData V2: A dataset for robot learning at scale,” in *Conference on Robot Learning (CoRL)*, 2023.
- [34] E. Collaboration, A. O’Neill, A. Rehman *et al.*, “Open X-Embodiment: Robotic learning datasets and RT-X models,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [35] InternVLA-M1 Contributors, “InternVLA-M1: A spatially guided vision-language-action framework for generalist robot policy,” *arXiv preprint arXiv:2510.13778*, 2025.
- [36] InternVLA-A1 Contributors, “InternVLA-A1: Unifying understanding, generation and action for robotic manipulation,” *arXiv preprint arXiv:2601.02456*, 2026.
- [37] Q. Lv, W. Kong, H. Li, J. Zeng *et al.*, “F1: A vision-language-action model bridging understanding and generation to actions,” *arXiv preprint arXiv:2509.06951*, 2025.
- [38] M. Reuss, M. Li, X. Jia, and R. Lioutikov, “Goal-conditioned imitation learning using score-based diffusion policies,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [39] Q. Lv, H. Li, X. Deng *et al.*, “Spatial-temporal graph diffusion policy with kinematic modeling for bimanual robotic manipulation,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 17 394–17 404.
- [40] X. Kang, Y. Shi, L. Liang, R. Gan, D. Liu, P. Zhang, D. Chen, X. Qin, Y. Zheng, J. Zheng *et al.*, “X-tokenizer: A multimodal action tokenizer for vision-language-action pretraining,” *arXiv preprint arXiv:2606.14752*, 2026.
- [41] C. Liu, X. Han, J. Gao, Y. Zhao, H. Chen, and Y. Du, “Oat: Ordered action tokenization,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2026.
- [42] N. M. M. Shafiullah, Z. J. Cui, A. Altanzaya, and L. Pinto, “Behavior transformers: Cloning k modes with one stone,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 22 955–22 968.
- [43] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, “Behavior generation with latent actions,” in *International Conference on Machine Learning (ICML)*, 2024, pp. 26 991–27 008.
- [44] H. Li, Q. Lv, R. Shao, X. Deng, Y. Li, J. Hao, and L. Nie, “Star: Learning diverse robot skill abstractions through rotation-augmented vector quantization,” in *International Conference on Machine Learning (ICML)*, 2026, pp. 35 562–35 580.
- [45] Y. Zhong, X. Liu, J. Wei *et al.*, “Vision-language-action models for robotics: A review towards real-world applications,” *IEEE Access*, vol. 13, pp. 162 467–162 504, 2025.

- [46] C. Liu, X. Han, J. Gao, Y. Zhao, H. Chen, and Y. Du, "Oat: Ordered action tokenization," 2026. [Online]. Available: <https://arxiv.org/abs/2602.04215>
- [47] K. Pertsch, K. Black, O. Mees *et al.*, "FAST: Efficient action tokenization for vision-language-action models," in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [48] A. J. Ijspeert, J. Nakanishi, and S. Schaal, "Movement imitation with nonlinear dynamical systems in humanoid robots," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2002.
- [49] A. Paraschos, C. Daniel, J. Peters, and G. Neumann, "Probabilistic movement primitives," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [50] S. Bahl, M. Mukadam, A. Gupta, and D. Pathak, "Neural dynamic policies for end-to-end sensorimotor learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [51] P. M. Scheikl, N. Schreiber, C. Haas, N. Freymuth, G. Neumann, R. Lioutikov, and F. Mathis-Ullrich, "Movement primitive diffusion: Learning gentle robotic manipulation of deformable objects," *IEEE Robotics and Automation Letters (RA-L)*, 2024.
- [52] F. Yang, P. Jing, K. Qu, N. Zhao, and Y. Su, "Abpolicy: Asynchronous b-spline flow policy for real-time and smooth robotic manipulation," 2026.
- [53] M. Tian, Y. Li, S. Liu, A. Ijspeert, and S. Calinon, "Spline policy: A structured representation for robot policies," *arXiv preprint arXiv:2606.07386*, 2026.
- [54] K. Black, A. Z. Ren, M. Equi, and S. Levine, "Training-time action conditioning for efficient real-time chunking," *arXiv preprint arXiv:2512.05964*, 2025.
- [55] Y. Zhao, L. Zhao, B. Cheng, G. Yao, X. Wen, and H. Gao, "Vla-rail: A real-time asynchronous inference linker for vla models and robots," *arXiv preprint arXiv:2512.24673*, 2025.
- [56] K. Sendai, M. Alvarez, T. Matsushima, Y. Matsuo, and Y. Iwasawa, "Leave no observation behind: Real-time correction for vla action chunks," *arXiv preprint arXiv:2509.23224*, 2025.
- [57] J. Ma, X. Chen, W. Bao, J. Xu, and H. Wang, "MADiff: Motion-aware mamba diffusion models for hand trajectory prediction on egocentric videos," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 48, pp. 3250–3267, 2025.
- [58] J. Ma, W. Bao, J. Xu, G. Sun, Y. Zheng, E. Zhang, X. Chen, and H. Wang, "Uni-hand: Universal hand motion forecasting in egocentric views," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, pp. 1–18, 2026.
- [59] Z. Li, Y. Zhong, R. Song, T. Li, L. Ma, and W. Zhang, "DeTAL: Open-vocabulary temporal action localization with decoupled networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 46, no. 12, pp. 7728–7741, 2024.
- [60] B. Wang, Y. Zhao, L. Yang, T. Long, and X. Li, "Temporal action localization in the deep learning era: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 46, no. 4, pp. 2171–2190, 2024.
- [61] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations (ICLR)*, 2023.
- [62] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *International Conference on Learning Representations (ICLR)*, 2023.
- [63] P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel, "Gello: A general, low-cost, and intuitive teleoperation framework for robot manipulators," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [64] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.
- [65] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu, "LIBERO-Plus: In-depth robustness analysis of vision-language-action models," *arXiv preprint arXiv:2510.13626*, 2025.
- [66] J. Cen, C. Yu, H. Yuan *et al.*, "WorldVLA: Towards autoregressive action world model," *arXiv preprint arXiv:2506.21539*, 2025.
- [67] Z. Hou, T. Zhang, Y. Xiong *et al.*, "Diffusion transformer policy: Scaling diffusion transformer for generalist vision-language-action learning," *arXiv preprint arXiv:2410.15959*, 2024.
- [68] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [69] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov *et al.*, "Paligemma: A versatile 3b vlm for transfer," *arXiv preprint arXiv:2407.07726*, 2024.
- [70] C. de Boor, *A Practical Guide to Splines*. Springer-Verlag, 1978.
- [71] G. Farin, *Curves and Surfaces for Computer-Aided Geometric Design: A Practical Guide*, 5th ed. Morgan Kaufmann, 2002.
- [72] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "Chomp: Gradient optimization techniques for efficient motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009.
- [73] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "Stomp: Stochastic trajectory optimization for motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011.
- [74] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *International Journal of Robotics Research (IJRR)*, vol. 33, no. 9, pp. 1251–1270, 2014.

APPENDIX A

PROOFS OF PROPOSITIONS 3.1–3.3

This appendix proves the three propositions of Sec. 3.4 and reports the spectrum of the Hermite operator underlying them (Table 13). Proposition 3.1 identifies the metric under which a trajectory-space loss supervises the boundary coordinates; Proposition 3.2 bounds the approximation error and the bandwidth of the scaffold; Proposition 3.3 bounds the seam discontinuity, with the position and velocity components treated separately, because only the latter requires the operator to be invertible on its range.

One convention holds throughout. The action space is delta end-effector control on every platform, so $\mathbf{a}^c$ is a sequence of per-step displacements and the boundary coordinates (p, v) of a Hermite segment correspond physically to endpoint velocities and accelerations of the executed motion. We retain the position/velocity notation for consistency with the spline literature; Remark A.4 states the translation to physical derivatives explicitly.

A.1 Notation and standing assumptions

All statements are given for one smoothly varying channel, i.e., $\boldsymbol{\theta} \in \mathbb{R}^{4K}$ and $\mathbf{a}^c \in \mathbb{R}^T$, and extend to the D_c channels column-wise. Throughout, f denotes the underlying displacement profile of the demonstration as a function of step index, so that $\|f''\|_\infty$ bounds the jerk of the executed motion in control-step units.

Recall from Eq. (9) that timestamp $i \in \{0, \dots, T-1\}$ is assigned to a unique segment $k(i)$ with local coordinate $u_i = (i - i_{k(i)})/s_{k(i)} \in [0, 1]$: timestamps $i \in [i_k, i_{k+1})$ belong to segment k , and the final timestamp $T-1$ belongs to segment $K-1$ with $u = 1$. Row i of $\mathbf{H}$ is supported on the four columns of segment $k(i)$ only. Hence the rows and the columns of $\mathbf{H}$ are both partitioned by segment: after a permutation of rows,

$$\mathbf{H} = \text{diag}(\mathbf{H}_0, \dots, \mathbf{H}_{K-1}), \quad \mathbf{H}_k \in \mathbb{R}^{m_k \times 4},$$

where m_k is the number of timestamps assigned to segment k , with $\sum_k m_k = T$ ($m_k = s_k$ for $k < K-1$ and $m_{K-1} = s_{K-1} + 1$ under the assignment above).

Assumption 1 (identifiability). Every segment contains at least four sampled timestamps, $m_k \geq 4$ for all k .

Assumption 1 necessarily requires $4K \leq T$, since $\sum_k m_k = T$. It is satisfied by the operative configuration $K = 2$ of the main paper for both $T = 10$ (segments of four and six timestamps) and $T = 50$ (at least 24). It is used only by Propositions 3.1 and 3.3, through Lemma A.2; Proposition 3.2 requires no rank assumption. Note that $m_k \geq 4$ is also *necessary* for $\mathbf{H}_k$ to have full column rank, since $\text{rank}(\mathbf{H}_k) \leq m_k$.

Lemma A.1 (elementary properties of the Hermite basis). For the basis functions of Eq. (8) and $u \in [0, 1]$:

(a) (Endpoint cardinality.)

	h_{00}	h_{10}	h_{01}	h_{11}
value at $u=0$	1	0	0	0
derivative at $u=0$	0	1	0	0
value at $u=1$	0	0	1	0
derivative at $u=1$	0	0	0	1

TABLE 13

Spectrum of the Hermite operator $\mathbf{H}$ under the rounded uniform grid of Eq. (9). Here m_k is the number of sampled timestamps per segment, and the column headed A. 1 marks whether Assumption 1 holds. Rows with $\sigma_{\min} = 0$ are column-rank deficient, so the boundary coordinates are not identifiable from the decoded trajectory. The operative configuration $K = 2$ is in bold.

K	$4K$	m_k	$\min_k m_k$	A. 1	$\sigma_{\min}$	$\sigma_{\max}$	cond
$T = 10$ (LIBERO, LIBERO-plus)							
1	4	(10)	10	✓	0.0937	2.2607	24.1
2	8	(4, 6)	4	✓	0.0189	1.7475	92.7
3	12	(3, 3, 4)	3	—	0	1.4230	∞
4	16	(2, 2, 3, 3)	2	—	0	1.3303	∞
$T = 50$ (real robot)							
1	4	(50)	50	✓	0.1669	5.0666	30.4
2	8	(24, 26)	24	✓	0.1056	3.6517	34.6
3	12	(16, 17, 17)	16	✓	0.0838	2.9645	35.4
4	16	(12, 12, 13, 13)	12	✓	0.0698	2.5976	37.2
5	20	(10, 10, 9, 10, 11)	9	✓	0.0564	2.3716	42.1
6	24	(8, 8, 8, 9, 8, 9)	8	✓	0.0510	2.1723	42.6

(b) (Affine reproduction.) $h_{00}(u) + h_{01}(u) \equiv 1$, and

$$\begin{aligned} h_{00}(u) - (1 - u) &= -(h_{01}(u) - u) \\ &= h_{10}(u) + h_{11}(u) = u(2u - 1)(u - 1). \end{aligned}$$

(c) (Extrema.) $0 \leq h_{10}(u) = u(1 - u)^2 \leq \frac{4}{27}$ (maximum at $u = \frac{1}{3}$), $-\frac{4}{27} \leq h_{11}(u) = -u^2(1 - u) \leq 0$ (minimum at $u = \frac{2}{3}$), and

$$h_{10}(u) + |h_{11}(u)| = u(1 - u)^2 + u^2(1 - u) = u(1 - u) \leq \frac{1}{4}.$$

$$\text{Moreover } \max_u |h_{00}| = \max_u |h_{01}| = 1.$$

(d) (Second derivatives.) $h''_{00} = 12u - 6$, $h''_{10} = 6u - 4$, $h''_{01} = -12u + 6$, $h''_{11} = 6u - 2$.

(e) (Third derivatives.) $h'''_{00} \equiv 12$, $h'''_{10} \equiv 6$, $h'''_{01} \equiv -12$, $h'''_{11} \equiv 6$.

(f) (First-derivative bounds.) $|h'_{00}(u)| = |h'_{01}(u)| = 6u(1 - u) \leq \frac{3}{2}$ and $|h'_{10}(u)|, |h'_{11}(u)| \leq 1$ for all $u \in [0, 1]$.

Proof. Direct computation from Eq. (8). $\square$

Lemma A.2 (rank and spectrum of $\mathbf{H}$). Under Assumption 1, every block $\mathbf{H}_k$ has full column rank; consequently $\mathbf{H}$ has full column rank and $\sigma_{\min}(\mathbf{H}) = \min_k \sigma_{\min}(\mathbf{H}_k) > 0$.

Proof. By Lemma A.1(a) the 4×4 matrix obtained by applying the four endpoint functionals (value/derivative at $u = 0, 1$) to $\{h_{00}, h_{10}, h_{01}, h_{11}\}$ is the identity, so the four basis functions are linearly independent and form a basis of the space $\mathcal{P}_3$ of cubic polynomials. Write $[h_j(u_i)]_{i \leq m_k, j \leq 4} = \mathbf{V}\mathbf{B}$, where $\mathbf{V} \in \mathbb{R}^{m_k \times 4}$ is the monomial matrix $\mathbf{V}_{il} = u_i^\ell$, $\ell = 0, \dots, 3$, and $\mathbf{B} \in \mathbb{R}^{4 \times 4}$ is the (invertible) change of basis from monomials to the Hermite basis. The nodes $\{u_i\}$ within one segment are pairwise distinct and $m_k \geq 4$; hence $\mathbf{V}\mathbf{c} = 0$ would force a nonzero cubic to vanish at $m_k \geq 4$ distinct points, so $\mathbf{V}$ has full column rank and so does $\mathbf{H}_k$. The block-diagonal structure implies that the singular values of $\mathbf{H}$ are the union of those of the blocks. $\square$

Table 13 reports the spectrum of $\mathbf{H}$ for both horizons used in the main paper. The knot grid uses round-half-to-even, the default of the Python/NumPy/JAX round primitive, so that $i_1 = 4$ at $T = 10$, $K = 2$; the script that produces the table is released with the code.

A.2 Proof of Proposition 3.1 (supervision geometry)

Let $\mathbf{P}_H = \mathbf{H}(\mathbf{H}^\top \mathbf{H})^{-1} \mathbf{H}^\top$ denote the orthogonal projector onto $\text{col}(\mathbf{H})$ and $\boldsymbol{\theta}_{\text{LS}}(\mathbf{a}) = (\mathbf{H}^\top \mathbf{H})^{-1} \mathbf{H}^\top \mathbf{a}$ (well defined by Lemma A.2). Decompose $\mathbf{a} = \mathbf{P}_H \mathbf{a} + (\mathbf{I} - \mathbf{P}_H) \mathbf{a}$ and note $\mathbf{P}_H \mathbf{a} = \mathbf{H} \boldsymbol{\theta}_{\text{LS}}(\mathbf{a})$. Then

$$\mathbf{H} \boldsymbol{\theta} - \mathbf{a} = \underbrace{\mathbf{H}(\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{LS}})}_{\in \text{col}(\mathbf{H})} - \underbrace{(\mathbf{I} - \mathbf{P}_H) \mathbf{a}}_{\perp \text{col}(\mathbf{H})}.$$

The two terms are orthogonal, so by the Pythagorean identity

$$\begin{aligned} \|\mathbf{H} \boldsymbol{\theta} - \mathbf{a}\|_2^2 &= \|\mathbf{H}(\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{LS}})\|_2^2 + \|(\mathbf{I} - \mathbf{P}_H) \mathbf{a}\|_2^2 \\ &= \|\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{LS}}\|_{\mathbf{H}^\top \mathbf{H}}^2 + \|(\mathbf{I} - \mathbf{P}_H) \mathbf{a}\|_2^2, \end{aligned}$$

which is Eq. (21). ■

Remark A.3 (scale disparity of the two metrics). By construction, the entries of the position columns of $\mathbf{H}$ are values of h_{00} or h_{01} and reach 1 (Lemma A.1(c)), whereas all entries of the velocity columns are values of h_{10} or h_{11} and are bounded in magnitude by $4/27$. An identity-metric loss on $\boldsymbol{\theta}$ therefore penalizes velocity errors at least $27/4 \approx 6.75$ times more strongly than their worst-case pointwise influence on the decoded trajectory, which the metric $\mathbf{H}^\top \mathbf{H}$ in Eq. (21) corrects automatically. The same fact explains the small values of $\sigma_{\min}(\mathbf{H})$ in Table 13; see Remark A.5.

A.3 Proof of Proposition 3.2 (expressivity and bandwidth)

Part (i). Fix a segment k and write $s = s_k$, $F(u) := f(i_k + us)$ for $u \in [0, 1]$, so that $F \in C^2([0, 1])$ and $\|F''\|_\infty = s^2 \|f''\|_{\infty, [i_k, i_{k+1}]}$. The target encoder of Eq. (10) yields

$$\begin{aligned} p_s &= F(0), & p_e &= F(1), \\ v_s &= s(f(i_{k+1}) - f(i_k)) = s(F(\tfrac{1}{s}) - F(0)), \\ v_e &= s(F(1) - F(1 - \tfrac{1}{s})), \end{aligned}$$

and the scaffold on this segment is $g(u) = h_{00}(u)F(0) + h_{10}(u)v_s + h_{01}(u)F(1) + h_{11}(u)v_e$.

Knot exactness. By Lemma A.1(a), $g(0) = F(0) = f(i_k)$ and $g(1) = F(1) = f(i_{k+1})$. Since the two copies of an interior knot in $\boldsymbol{\theta}_{\text{gt}}$ read the same ground-truth frame (Alg. 1), the piecewise scaffold is continuous and exact at every knot.

Uniform error. By the mean value theorem, $v_s = F'(\xi_s)$ with $\xi_s \in (0, \frac{1}{s})$, $v_e = F'(\xi_e)$ with $\xi_e \in (1 - \frac{1}{s}, 1)$, and the chord slope satisfies $\Delta F := F(1) - F(0) = F'(\eta)$ with $\eta \in (0, 1)$. Let $\ell(u) = (1 - u)F(0) + uF(1)$ be the linear interpolant. Using Lemma A.1(b),

$$\begin{aligned} g(u) - \ell(u) &= [h_{00}(u) - (1 - u)]F(0) + [h_{01}(u) - u]F(1) \\ &\quad + h_{10}(u)v_s + h_{11}(u)v_e \\ &= [h_{10}(u) + h_{11}(u)](F(0) - F(1)) \\ &\quad + h_{10}(u)v_s + h_{11}(u)v_e \\ &= h_{10}(u)(v_s - \Delta F) + h_{11}(u)(v_e - \Delta F). \end{aligned}$$

Applying the mean value theorem to F' between ξ_s (resp. ξ_e) and η gives $|v_s - \Delta F| \leq \|F''\|_\infty$ and $|v_e - \Delta F| \leq \|F''\|_\infty$, hence by Lemma A.1(c)

$$|g(u) - \ell(u)| \leq (h_{10}(u) + |h_{11}(u)|) \|F''\|_\infty \leq \tfrac{1}{4} \|F''\|_\infty.$$

Combining with the classical linear-interpolation remainder $|F(u) - \ell(u)| = \frac{1}{2} u(1 - u) |F''(\zeta_u)| \leq \frac{1}{8} \|F''\|_\infty$, the triangle inequality yields, for every $u \in [0, 1]$,

$$\begin{aligned} |F(u) - g(u)| &\leq \left(\tfrac{1}{8} + \tfrac{1}{4}\right) \|F''\|_\infty \\ &= \tfrac{3}{8} s^2 \|f''\|_\infty \leq \tfrac{3}{8} s_{\max}^2 \|f''\|_\infty, \end{aligned}$$

where time is measured in control steps. Taking the maximum over segments proves part (i). The two contributions are the classical linear-interpolation remainder (1/8) and the error incurred by estimating the endpoint tangents with one-sided finite differences (1/4); the latter is the price of the encoder of Eq. (10) and would vanish for exact endpoint derivatives, which is why the rate is $O(s^2 \|f''\|_\infty)$ rather than the $O(s^4 \|f'''\|_\infty)$ of Hermite interpolation with exact tangents. No rank assumption is used, and the argument is valid for every $s_k \geq 1$ (for $s_k = 1$ both one-sided differences coincide with the chord). ■

Part (ii). On segment k the scaffold is a cubic polynomial in u , so its second derivative in u is affine and its third derivative is constant; by Lemma A.1(e),

$$\frac{d^3 g}{du^3} = 12(p_s^{(k)} - p_e^{(k)}) + 6(v_s^{(k)} + v_e^{(k)}).$$

With the time change $t = i_k + u s_k$ (hence $d/dt = s_k^{-1} d/du$), the second derivative of $\mathbf{a}^c$ is piecewise linear in t and the third is piecewise constant with at most K distinct values,

$$\frac{d^3 \mathbf{a}^{(k)}}{dt^3} = \frac{12(p_s^{(k)} - p_e^{(k)}) + 6(v_s^{(k)} + v_e^{(k)})}{s_k^3},$$

so with the per-step endpoint velocities $\nu^{(k)} = v^{(k)}/s_k$ the triangle inequality gives exactly the bound of Eq. (22). Under the rounded uniform grid, $s_k \approx (T - 1)/K$, so the admissible ceiling scales as K^3 while, by part (i), the scaffold approximation error of the target decays as K^{-2} ; the two bounds pull in opposite directions, predicting an interior optimum in K . ■

Remark A.4 (physical derivatives in a displacement action space). The action channels are per-step displacements, so the executed position is the running sum of $\mathbf{a}^c$ and the jerk measured in Sec. 4.5 is the *second* derivative of $\mathbf{a}^c$. Part (ii) characterizes derivatives of $\mathbf{a}^c$ itself; translating, a scaffold in the span of $\mathcal{D}_H$ yields a piecewise *quartic* position trajectory whose jerk is piecewise *linear* with at most K pieces. Explicitly, by Lemma A.1(d), on segment k ,

$$\left. \frac{d^2 \mathbf{a}^{(k)}}{dt^2} \right|_u = \frac{c_1^{(k)} u - c_0^{(k)}}{s_k^2}, \quad \begin{aligned} c_1^{(k)} &= 12\Delta p^{(k)} + 6(v_s^{(k)} + v_e^{(k)}), \\ c_0^{(k)} &= 6\Delta p^{(k)} + 4v_s^{(k)} + 2v_e^{(k)}, \end{aligned}$$

with $\Delta p^{(k)} = p_s^{(k)} - p_e^{(k)}$. This is affine in u and therefore attains its extrema at the endpoints, giving

$$\sup_t \left| \frac{d^2 \mathbf{a}^{(k)}}{dt^2} \right| \leq \frac{6|\Delta p^{(k)}|}{s_k^2} + \frac{4(|v_s^{(k)}| + |v_e^{(k)}|)}{s_k}.$$

The measured jerk ceiling thus scales as K^2 while the approximation error of part (i) decays as K^{-2} , so the trade-off described above is unaffected. The piecewise-linear structure with only $K = 2$ pieces is what structurally precludes the isolated transient spikes visible in Fig. 6.

A.4 Proof of Proposition 3.3 (seam control)

Step 1 (handover states as linear functionals). Let $W \leq T$ denote the execution window, so that the last executed timestep of a chunk is $W-1$, lying in segment $k^* = k(W-1)$ with local coordinate $u^* = (W-1 - i_{k^*})/s_{k^*}$. On the *incoming* side of a handover, the first action of the new chunk is its timestep 0, i.e., $u = 0$ of segment 0; by Lemma A.1(a) the scaffold position and velocity there are exactly the boundary coordinates $\hat{p}_s^{(0)}$ and $\hat{v}_s^{(0)}$ of $\hat{\theta}'$, independently of W . On the *outgoing* side, the scaffold position and velocity at timestep $W-1$ are the fixed linear functionals $\mathbf{r}_p^\top \hat{\theta}$ and $\mathbf{r}_v^\top \hat{\theta}$, where $\mathbf{r}_p$ is row $W-1$ of $\mathbf{H}$ (supported on block k^* with entries $h_\bullet(u^*)$) and $\mathbf{r}_v$ tabulates $h'_\bullet(u^*)/s_{k^*}$ on the same block. If $W-1$ coincides with a segment knot, then $u^* \in \{0, 1\}$ and, by Lemma A.1(a), $\mathbf{r}_p$ and $\mathbf{r}_v$ are coordinate selectors (up to the fixed scaling $1/s_{k^*}$ for velocity): the outgoing state reduces to a boundary-coordinate pair of $\hat{\theta}$ as well, in particular to $(\hat{p}_e^{(K-1)}, \hat{v}_e^{(K-1)})$ under full-chunk execution ($W = T$).

Step 2 (position: a trajectory functional). Because $\mathbf{r}_p$ is row $W-1$ of $\mathbf{H}$, the outgoing scaffold position is itself a trajectory component, $\mathbf{r}_p^\top \hat{\theta} = (\mathbf{H}\hat{\theta})_{W-1}$, and by Lemma A.1(a) the incoming one is $\hat{p}_s^{(0)} = (\mathbf{H}\hat{\theta}')_0$. Inserting the two demonstration frames and applying the triangle inequality,

$$|\mathbf{r}_p^\top \hat{\theta} - \hat{p}_s^{(0)}| \leq \|\mathbf{H}\hat{\theta} - \mathbf{a}\|_\infty + \Delta_p^{\text{demo}} + \|\mathbf{H}\hat{\theta}' - \mathbf{a}'\|_\infty, \quad (23)$$

where $\Delta_p^{\text{demo}} = |\mathbf{a}[W-1] - \mathbf{a}'[0]|$ is the literal across-seam step of the demonstration. Equation (23) involves neither $\sigma_{\min}(\mathbf{H})$ nor $\|\mathbf{r}_p\|_2$, and holds for every window W without requiring knot alignment: the position component of a seam is controlled directly by the trajectory-space residuals that Eqs. (12), (17) and (19) minimize.

Step 3 (velocity: a coordinate functional). The row $\mathbf{r}_v$ tabulates $h'_\bullet$, which is not a row of $\mathbf{H}$, so the velocity component must pass through the coordinates. By the identity of Proposition 3.1, for any target $\mathbf{a}$,

$$\|\mathbf{H}\hat{\theta} - \mathbf{a}\|_2^2 \geq \|\mathbf{H}(\hat{\theta} - \theta_{\text{LS}}(\mathbf{a}))\|_2^2 \geq \sigma_{\min}(\mathbf{H})^2 \|\hat{\theta} - \theta_{\text{LS}}(\mathbf{a})\|_2^2,$$

using Lemma A.2, hence $\|\hat{\theta} - \theta_{\text{LS}}(\mathbf{a})\|_2 \leq \|\mathbf{H}\hat{\theta} - \mathbf{a}\|_2 / \sigma_{\min}(\mathbf{H})$. Writing j' for the index of $v_s^{(0)}$ within θ' and using $|\mathbf{r}^\top \mathbf{x}| \leq \|\mathbf{r}\|_2 \|\mathbf{x}\|_2$ together with $|x_{j'}| \leq \|\mathbf{x}\|_2$,

$$|\mathbf{r}_v^\top \hat{\theta} - \hat{v}_s^{(0)}| \leq \|\mathbf{r}_v\|_2 \frac{\|\mathbf{H}\hat{\theta} - \mathbf{a}\|_2}{\sigma_{\min}(\mathbf{H})} + \Delta_v^{\text{demo}} + \frac{\|\mathbf{H}\hat{\theta}' - \mathbf{a}'\|_2}{\sigma_{\min}(\mathbf{H})}, \quad (24)$$

with Δ_v^{demo} the across-seam velocity increment of the demonstration in least-squares coordinates. By Lemma A.1(c) and (f), the functional row norms satisfy

$$\|\mathbf{r}_p\|_2 \leq \sqrt{1 + 2\left(\frac{4}{27}\right)^2} < 1.03, \quad \|\mathbf{r}_v\|_2 \leq \frac{\sqrt{13/2}}{s_{k^*}} \approx \frac{2.55}{s_{k^*}},$$

so the velocity functional is contractive whenever $s_{k^*} \geq 3$. Together, Eqs. (23) and (24) bound the seam discontinuity in position and velocity by the prediction errors of the two adjacent chunks plus the across-seam increment intrinsic to the demonstration, which proves the claim. ■

Remark A.5 (magnitude of the amplification factor). The factor $1/\sigma_{\min}(\mathbf{H})$ appearing in Eq. (24) equals 53.0 at

$T = 10, K = 2$ and 9.5 at $T = 50, K = 2$ (Table 13). Its size has the same origin as Remark A.3: the right singular direction attaining $\sigma_{\min}$ places 97.6% of its mass on velocity coordinates, whose columns are bounded by $4/27$ and therefore influence the decoded trajectory only weakly. The velocity bound is consequently loose, and markedly looser at the simulation horizon than at the real-robot horizon at which the seam measurements of Table 12 are taken. The position bound of Eq. (23) is free of this factor.

Remark A.6 (discrete variant). For HERMITE-VLA_{DH} the deployed $\hat{\theta}$ is dequantized (Eq. (13)); uniform quantization with bin width Δ_j per coordinate perturbs the residual by at most an additive $\|\mathbf{H}\|_2 \frac{1}{2} (\sum_j \Delta_j^2)^{1/2}$, so Eqs. (23) and (24) hold with this extra additive term.

Corollary A.7 (knot-aligned and full-chunk execution). When $W-1 \in \{i_1, \dots, i_K\}$, both sides of every handover are exactly boundary-coordinate pairs of the respective $\hat{\theta}$, and Eq. (24) holds with $\|\mathbf{r}_v\|_2 = 1/s_{k^*}$. In the protocols of the main paper, the LIBERO window $W = 5$ satisfies $W-1 = 4 = i_1$ on the rounded uniform grid ($T = 10, K = 2$), so the simulated seams are knot-aligned, whereas the real-robot window $W = 20$ places $W-1 = 19$ strictly inside the first segment ($T = 50, K = 2$) and is covered by the general statement. Equation (23) is insensitive to this distinction.

Remark A.8 (encoder targets instead of least-squares targets). In Eq. (24), replacing θ_{LS} by the encoder targets θ_{gt} of Alg. 1 makes the demonstrator increment literal, at the price of one extra additive term $\|\mathbf{H}\theta_{\text{gt}} - \mathbf{a}\|_2 / \sigma_{\min}(\mathbf{H})$ per chunk, bounded via Proposition 3.2(i) by $\sqrt{T} \frac{3}{8} s_{\max}^2 \|f''\|_\infty / \sigma_{\min}(\mathbf{H})$. Equation (23) already uses the literal increment.

Remark A.9 (identifiability ceiling and the K -ablation). At $T = 10$, Assumption 1 forces $K \leq 2$: already at $K = 3$ the operator has more columns than rows ($4K = 12 > T$), and two of its three blocks satisfy $m_k = 3 < 4$, so $\mathbf{H}$ is column-rank deficient and $\sigma_{\min}(\mathbf{H}) = 0$ (Table 13). The boundary coordinates are then unidentifiable from the decoded trajectory and the velocity guarantee of Eq. (24) degenerates, although training remains well-posed because the objectives of Eqs. (17) and (19) are posed in trajectory space and Eq. (23) is unaffected. The empirically optimal configuration $K = 2$ of Table 6 therefore coincides with the largest identifiable segment count at this horizon, which we regard as a complementary explanation of the degradation observed at $K = 3, 4$ alongside the bias-fidelity trade-off of Proposition 3.2. At the real-robot horizon $T = 50$ the constraint is inactive for all $K \leq 12$, so there $K = 2$ reflects the trade-off rather than an identifiability limit; we did not sweep K at $T = 50$, and note this in Sec. 5.